UNIVERSITI POLY-TECH MALAYSIA
UPTM

Name:
1. NURUL FADILAH BINTI SHARIZAL
2. NURBASYIERAH BALQIS BINTI HAZIZAN
3. NURUL SYAKIRIN AHMAD
4. MUHAMMAD SYAMIR BIN MOHD NORZALIMI
5. MUHAMMAD ARIEF AZIM BIN RASIS FARID

ID Number:
1. AM2512021472
2. AM2512021567
3. AM2512021352
4. AM2512021746
5. AM2512021797

Lecturer:SITI ROBAYA BINTI JANTAN		Lab group / Tutorial group / Tutor (if applicable) **SECTION 2**
Course and Course Code: WEB API DEVELOPMENT SWC3633/SWC4443		Submission Date: **28/8/2026**
Assignment No. / Title: **PROJECT**		Extension & Late submission: ALLO WED / **DISALLOWED**
Assignment type: **GROUPING**	% of Assignment **30 Mark**	Returning Date:

Penalties:
1. 40% of the original mark will be deducted for every one-week period after the submission date
2. No work will be accepted after two weeks of the deadline
3. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension
4. Extension will not exceed one week

Declaration: I the undersigned confirm that I have read and agreed to abide by these regulations on plagiarism and cheating. I confirm that this piece of work is my own. I consent to appropriate storage of my work for checking to ensure that there is no plagiarism/ academic cheating.

Signature(s):

Full Name:
This section may be used for feedback or other information:

Table of Contents

1.0 SYSTEM OVERVIEW 5
1.1 System Description 5
1.2 System Objectives 6
1.3 Target Users 6
1.4 Main Features 7
2.0 Database Design 8
2.1 Overview 8
2.2 Entity Relationship Diagram (ERD) 8
Relationships 9
2.3 Data Dictionary 9
users 9
venues 9
events 10
bookings 10
2.4 SQL Scripts (DDL and DML) 11
2.5 Design Decisions 11
3.0 RESTful API Documentation 15
3.1 API Base URL 15
3.2 Endpoints and HTTP Methods 15
3.3 Authentication Requirements 16
3.4 Request JSON 16
3.5 Response JSON 17
HTTP Status Codes 18
4.0 API Testing 19
4.1 Testing Environment 19
4.2 Test Cases and Results 19
4.3 Testing Evidence 21
4.4 Exported Postman Collection 28
5.0 SYSTEM IMPLEMENTATION 29
5.1 Shared Backend Implementation 29
5.2 Frontend Implementation 31
5.3 Frontend and API Connection 34
6.0 SYSTEM DEMONSTRATION 38
6.1 Member 1: Event Browsing and Public Discovery Portal 38
6.2 Member 2: Authentication and User Profile Management 39
6.3 Member 3: Event Management Dashboard 40
6.4 Member 4: Venue Management Dashboard 46
6.5 Member 5: Booking Management 48
7.0 Project Repository 49
7.1 Repository Information 49
7.2 Repository Structure 49
7.3 Setup Instructions 49

7.4 Git Workflow 49
Commands used to create the repository (Member 2) 49
7.5 Contribution Record 50
Appendix A – schema.sql (DDL) 51
Appendix B – seed.sql (DML) 53
8 Additional and Advanced Features 54
8.1 Event Search, Filtering, and Pagination 54
8.2 Role-Based Access and Ownership 54
8.3 Venue Availability and Schedule Conflict Checks 54
8.4 Input Validation and Booking-Safe Deletion 54
8.5 Event Management Frontend 54
8.6 API Verification Evidence 55
9.0 Debugging and Performance Optimisation 56
9.2 Performace Testing 57
10.0 Individual Reflection and Contribution Record 59
10.1 Nurbasyierah Reflective Report 59
10.2 Nur Fadilah Reflective Report 60
10.3 NurSyakirin Reflective Report 61
10.4 Muhammad Syamir Reflective Report 62
10.5 Muhammad Arief Azim Reflective Report 63

1.0 SYSTEM OVERVIEW

1.1 System Description

The Event Ticketing and Venue Booking System, presented through the EventHub web interface, is a web-based application designed to support the management and discovery of events, venues, ticket availability and customer bookings. The system provides a central platform for administrators and organisers to manage event information and venue details, while allowing customers to browse upcoming events, view relevant information and monitor ticket availability.

The proposed application follows Representational State Transfer (REST) principles. Its frontend communicates with a shared backend through RESTful Web API endpoints, and the system exchanges data in JavaScript Object Notation (JSON) format. The main resources include users, events, venues and bookings. Authentication, role-based access control, input validation and error handling are included in the project requirements to support reliable and secure operations. Together, these functions aim to simplify event-related processes and make event information more accessible to users.

1.2 System Objectives

The objectives of the proposed system are as follows:

1. To develop a RESTful Web API that supports the management of users, events, venues and customer bookings.

2. To provide a centralised platform for administrators and organisers to maintain event details, schedules, venues and ticket availability.

3. To enable customers to browse upcoming events, view event information, make or cancel bookings and check booking status.

4. To apply authentication, role-based access control, data validation and appropriate error handling to the relevant system operations.

5. To connect individual frontend interfaces to the shared API and extend system functionality through at least one genuine third-party API integration.

1.3 Target Users

The system is intended for the following user roles:

1. Administrator: The administrator manages user accounts and oversees the information and operations of the event ticketing and venue booking system. Access to administrative functions is controlled by the assigned user role.

2. Organiser / Staff: Organisers or authorised staff members create and maintain events, venue information, schedules and ticket availability according to their permissions.

3. Customer: Customers browse upcoming events, view event and venue details, make or cancel bookings and check their booking information and status.

1.4 Main Features

The principal features specified for the system are described below:

- User Management and Access Control: User records are managed through the API, with authentication and role-based permissions for administrators, organisers or staff, and customers.

- Event Management: Authorised users can create, view, update and manage events, including their basic information and schedules.

- Venue Management: The system maintains venue information and supports the assignment of suitable venues to events.

- Event Browsing and Search: Customers can browse event listings and view event details. The EventHub browsing frontend provides search, category and availability filtering, and sorting functions.

- Ticket Availability and Booking Management: Customers can view ticket availability and submit bookings, while the system maintains booking details and status.

- Booking Cancellation and Status Tracking: Customers can view their booking information and cancel reservations according to system rules.

- RESTful API Communication: Frontend interfaces communicate with the shared backend through JSON-based resource endpoints such as /users, /events, /venues and /bookings, using appropriate HTTP methods.

- Security, Validation and External Integration: The project includes request validation, centralised error handling, middleware or API management features, and at least one genuine external API service, such as a QR-code or email notification service.

2.0 Database Design

2.1 Overview

The system uses a MySQL database named event_ticketing_db (InnoDB engine, utf8mb4 character set) that runs on XAMPP. It contains four related tables: Users, Venues, Events and Bookings. Foreign keys enforce referential integrity. ON DELETE RESTRICT stops a user, venue or event from being deleted while other records still refer to it, so booking history is never lost by accident. In that situation the API returns a clear 409 Conflict message instead of a raw database error.

The design supports all three required roles. The Users table holds a role column (admin, organiser, customer) so that one login system can serve the Administrator, Organiser/Staff and Customer functions.

2.2 Entity Relationship Diagram (ERD)

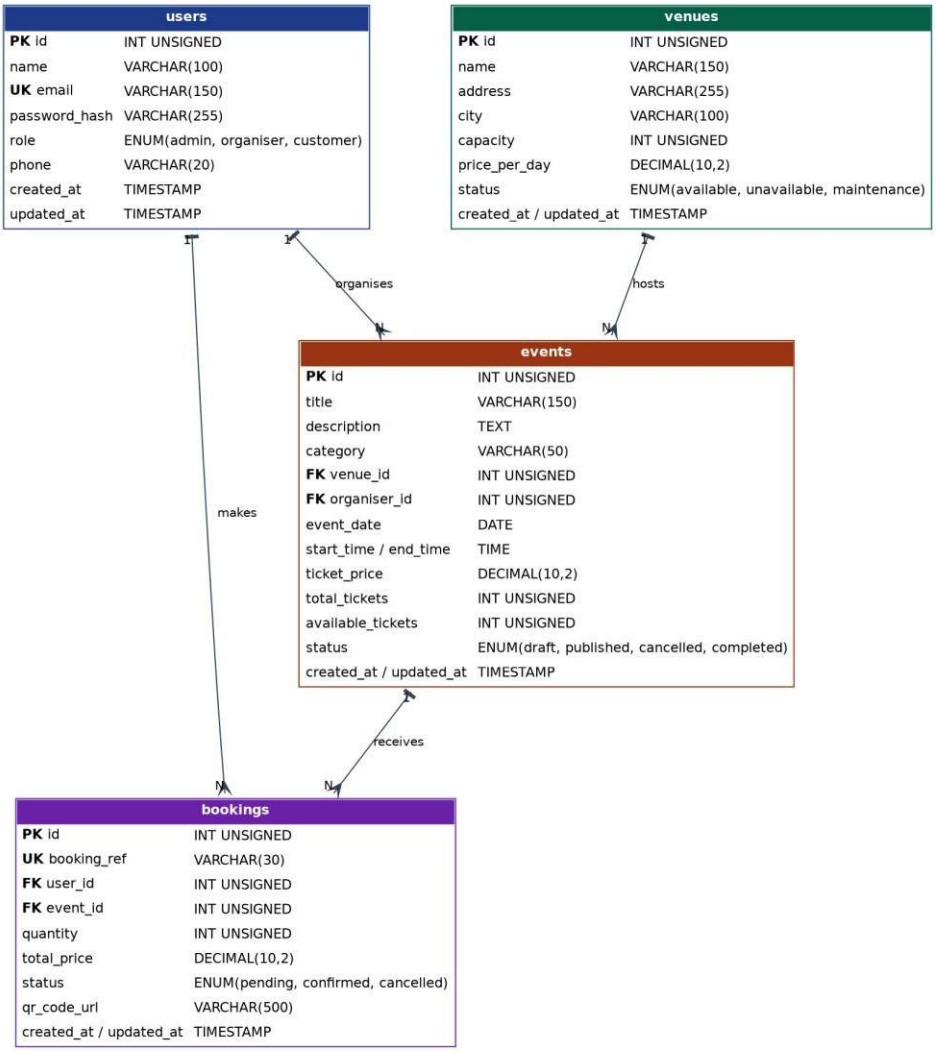


Figure 1: ERD of event_ticketing_db (PK = primary key, FK = foreign key, UK = unique key). A full-size copy is in docs/ERD.png.

Relationships

Relationship	Type	Foreign key
A venue hosts events	One-to-many	events.venue_id → venues.id
A user (organiser) organises events	One-to-many	events.organiser_id → users.id
A user (customer) makes bookings	One-to-many	bookings.user_id → users.id
An event receives bookings	One-to-many	bookings.event_id → events.id

Users and Events are therefore linked in a many-to-many way through the Bookings table, which acts as the junction table and also stores the quantity, total price and status of each booking.

2.3 Data Dictionary

users

Column	Type	Constraints / notes
id	INT UNSIGNED	Primary key, auto increment
name	VARCHAR(100)	Required
email	VARCHAR(150)	Required, UNIQUE (used to log in)
password_hash	VARCHAR(255)	Required. bcrypt hash, never plain text
role	ENUM	admin, organiser, customer. Default 'customer'
phone	VARCHAR(20)	Optional
created_at / updated_at	TIMESTAMP	Set automatically

venues

Column	Type	Constraints / notes
id	INT UNSIGNED	Primary key, auto increment
name	VARCHAR(150)	Required
address / city	VARCHAR(255) / VARCHAR(100)	Required
capacity	INT UNSIGNED	Required, CHECK capacity > 0
price_per_day	DECIMAL(10,2)	Default 0.00
status	ENUM	available, unavailable, maintenance. Default 'available'

Column	Type	Constraints / notes
created_at / updated_at	TIMESTAMP	Set automatically

events

Column	Type	Constraints / notes
id	INT UNSIGNED	Primary key, auto increment
title / description	VARCHAR(150) / TEXT	Title required, description optional
category	VARCHAR(50)	Required. Default 'Community' (for example Music, Technology, Sports)
venue_id	INT UNSIGNED	Foreign key → venues.id (RESTRICT)
organiser_id	INT UNSIGNED	Foreign key → users.id (RESTRICT)
event_date	DATE	Required, indexed
start_time / end_time	TIME	Required
ticket_price	DECIMAL(10,2)	Default 0.00
total_tickets / available_tickets	INT UNSIGNED	CHECK available_tickets <= total_tickets
status	ENUM	draft, published, cancelled, completed. Default 'draft', indexed
created_at / updated_at	TIMESTAMP	Set automatically

bookings

Column	Type	Constraints / notes
id	INT UNSIGNED	Primary key, auto increment
booking_ref	VARCHAR(30)	Required, UNIQUE (for example BK-20261001-0001)
user_id	INT UNSIGNED	Foreign key → users.id (RESTRICT), indexed
event_id	INT UNSIGNED	Foreign key → events.id (RESTRICT), indexed
quantity	INT UNSIGNED	CHECK quantity > 0
total_price	DECIMAL(10,2)	Required
status	ENUM	pending, confirmed, cancelled. Default 'pending'
qr_code_url	VARCHAR(500)	Link generated by the external QR Code API

Column	Type	Constraints / notes
created_at / updated_at	TIMESTAMP	Set automatically

2.4 SQL Scripts (DDL and DML)

- DDL – database/schema.sql: creates the database, drops old tables safely, then creates all four tables with primary keys, foreign keys, UNIQUE constraints, CHECK constraints and indexes.
- DML – database/seed.sql: inserts 6 users, 5 venues, 6 events and 8 bookings (the minimum required is 5 records per table). The available_tickets values in events already match the confirmed bookings.
- Import order: schema.sql first, then seed.sql. The complete scripts are listed in Appendix A and Appendix B.

2.5 Design Decisions

1. One Users table with a role column keeps authentication simple and supports the three required roles.
2. Passwords are hashed with bcrypt (PHP password_hash) so that no plain-text password is stored.
3. ON DELETE RESTRICT protects booking history. The API converts the database error into a 409 Conflict response.
4. A unique booking reference and a QR code URL give every booking a scannable ticket identity.
5. Indexes on event_date, status, user_id and event_id speed up searching and filtering.
6. CHECK constraints (capacity > 0, quantity > 0, available_tickets <= total_tickets) stop invalid data at database level, in addition to API validation.

Users API, Login and JWT Authentication (Member 2 coding)

Authentication flow

7. The client sends POST /auth/login with email and password.
8. The server finds the user and checks the password with password_verify(). The same 401 message is used for an unknown email and a wrong password, so it does not reveal which one failed.
9. The server returns a signed JWT (HS256) containing sub (user id), role, name, email, iat and exp (valid for 2 hours).
10. The client sends Authorization: Bearer <token> on every protected request.
11. Middleware in core/auth.php verifies the signature and expiry (require_auth) and then checks the role (require_role).

Endpoints

Method	URL	Access	Description
POST	/auth/register	Public	Register a customer account
POST	/auth/login	Public	Log in and receive a JWT
GET	/auth/me	Logged-in user	View own profile
GET	/users	Admin	List users (page, limit, search, role)
GET	/users/{id}	Admin or self	View one user
POST	/users	Admin	Create a user with any role
PUT	/users/{id}	Admin or self	Update a user (only admin can change role)
DELETE	/users/{id}	Admin	Delete a user (not yourself)

Users API permissions

Action	Admin	Organiser	Customer
List users	Allowed	403 Forbidden	403 Forbidden
View user	Any user	Own profile only	Own profile only
Create user	Allowed	403 Forbidden	403 (use /auth/register)
Update user	Any user, including role	Own profile, no role change	Own profile, no role change
Delete user	Allowed (not self)	403 Forbidden	403 Forbidden

Validation and error handling

Name must be 2–100 characters, email must be valid, password must be at least 8 characters with letters and numbers, role must be admin, organiser or customer, and phone is optional but must match a phone format. Public registration can only create customers, which prevents privilege escalation. All errors use one JSON format:

```
{ "success": false, "message": "Validation failed.", "errors": { "email": "Email format is invalid." } }
```

Status	Meaning in this API
200 / 201	Request succeeded / record created
400	Bad request (invalid JSON, empty body, invalid ID, removing own admin role)
401	Missing, invalid or expired token, or wrong login details

Status	Meaning in this API
403	Role not allowed to perform the action
404	User or endpoint not found
409	Duplicate email, or user still has events or bookings
422	Validation failed (field-level errors returned)
500	Unexpected server or database error

Extra features

- Pagination (page and limit, maximum 100 per page) with total and total_pages returned.
- Search by name or email, and filter by role.
- Consistent JSON response and error format shared with the other members.
- CORS headers so every member's frontend can call the API.

Sample accounts

Role	Email	Password	User ID
Admin	admin@event.com	Admin@123	1
Organiser	hafiz@event.com	Organiser@123	2
Organiser	meiling@event.com	Organiser@123	3
Customer	daniel@mail.com	Customer@123	4
Customer	nurul@mail.com	Customer@123	5
Customer	arjun@mail.com	Customer@123	6

Test cases for Postman

#	Request	Expected result
1	POST /auth/login with valid admin account	200 and a token
2	POST /auth/login with wrong password	401
3	POST /auth/login with empty fields	422
4	POST /auth/register with a new customer	201
5	POST /auth/register with a duplicate email	409
6	GET /users without a token	401
7	GET /users as customer	403
8	GET /users?page=1&limit=3&search=a as admin	200 with pagination

#	Request	Expected result
9	GET /users/999 as admin	404
10	PUT /users/4 as customer with {"role":"admin"}	403
11	PUT /users/4 as customer with {"phone":"011-1234567"}	200
12	DELETE /users/2 as admin (user has events)	409
13	DELETE /users/{newly created id} as admin	200

3.0 RESTful API Documentation

Member 3 implemented the Events API and the event management frontend. The Events API is a resource handled by the shared PHP router at api/index.php, uses the shared MySQL database, and returns the project's standard JSON response format. The frontend calls the API to browse, create, view, update, and delete events.

3.1 API Base URL

For the shared XAMPP setup, the API base URL is:

```
http://localhost/event-ticketing/api
```

The frontend defaults to this URL. If the project is installed under a different folder, the API URL can be changed using the frontend's API configuration control. When the frontend is served with PHP's built-in server, Apache and MySQL must still be running to serve the API and database.

3.2 Endpoints and HTTP Methods

Method	Endpoint	Authentication and access	Description
GET	/events	Public for published events. Admin and organiser tokens can view every status.	List events; supports pagination and filters.
GET	/events/{id}	Public for published events. Admin and organiser tokens can view every status.	Retrieve an event by ID.
POST	/events	Admin or organiser	Create an event.
PUT	/events/{id}	Admin or the event's owning organiser	Update an event.
PATCH	/events/{id}	Admin or the event's owning organiser	Partially update an event.
DELETE	/events/{id}	Admin or the event's owning organiser	Delete an event if no booking references prevent deletion.

The list endpoint supports these query parameters:

Parameter	Purpose
page	Page number; defaults to 1.
limit	Results per page; defaults to 10 and is capped at 100.
search	Search event title and description.
category	Filter by category.
venue_id	Filter by venue ID.
status	Filter by draft, published, cancelled, or completed.
organiser_id	Filter by organiser ID.

Example request:

```
GET /event-ticketing/api/events?page=1&limit=10&search=music&status=published
```

3.3 Authentication Requirements

Event reads are public for published events. If a guest or customer requests an unpublished event by ID, the API returns 404. Admins and organisers can view unpublished events for management.

Creating, updating, and deleting events requires a valid JWT. The client sends it in this header:

```
Authorization: Bearer <JWT>
```

The JWT is obtained from the shared POST /auth/login endpoint. The Events API uses the shared require_auth() and require_role() middleware. Admins can manage any event; organisers can manage only events they own. Only admins can assign or reassign an event to a different organiser.

3.4 Request JSON

The following example creates a published event. title, venue_id, event_date, start_time, end_time, and total_tickets are required. description, category, ticket_price, available_tickets, and status are optional. Category defaults to Community, ticket price to 0, available tickets to the total, and status to draft.

```
POST /event-ticketing/api/events Content-Type: application/json Authorization: Bearer <JWT>

{
"title": "Johor Music Night",
"description": "An evening of live music.", "category": "Music",
"venue_id": 1,
"event_date": "2026-11-20",
"start_time": "19:00",
"end_time": "22:00",
"ticket_price": 35.00,
"total_tickets": 300,
"available_tickets": 300, "status": "published"
}
```

An administrator may also include organiser_id to assign the event to an existing admin or organiser. Other roles cannot assign events to another user.

For PUT or PATCH, send one or more supported event fields in the JSON body. The event ID is part of the URL. DELETE needs no request body.

3.5 Response JSON

Successful responses use the shared envelope. A successful create returns HTTP 201 and the event data:


```
{
"success": true,
"message": "Event created.", "data": {
"id": 12,
"title": "Johor Music Night", "category": "Music", "venue_id": 1, "venue_name": "City Hall",
"venue_city": "Johor Bahru", "organiser_id": 2,
"organiser_name": "Hafiz Ahmad", "event_date": "2026-11-20",
"start_time": "19:00:00",
"end_time": "22:00:00",
"ticket_price": "35.00",
"total_tickets": 300,
"available_tickets": 300, "status": "published"
}
}
```


The list response places event records in data and includes pagination metadata:


```
{
"success": true,
"message": "Events retrieved.", "data": [],
"pagination": { "page": 1,
"limit": 10,
"total": 0,
"total_pages": 0
}
}
```


Validation errors use HTTP 422 and identify invalid fields:


```
{
"success": false,
"message": "Validation failed.", "errors": {
"end_time": "End time must be later than start time."
}
}
```

HTTP Status Codes

Code	Meaning in the Events API
200	Event list or detail returned, event updated, or event deleted.
201	Event created.
400	Invalid event ID or empty update request.
401	Missing, invalid, or expired JWT on a protected request.
403	Role or event ownership does not permit the requested action.
404	Event or venue not found, or an unpublished event is hidden from a guest/customer.
405	HTTP method is not supported for the endpoint.
409	Venue unavailable, event schedule overlaps, or bookings prevent deletion.
422	Request fields fail validation.
500	Unexpected server error, returned through the shared exception handler.

4.0 API Testing

API testing was conducted using Postman to verify the functionality of the Venues API in the Event Ticketing and Venue Booking System. Testing covered creating, retrieving, updating and deleting venues, as well as checking venue availability. Both successful requests and error scenarios were tested to verify input validation, JWT authentication, role-based access control and protection of venues linked to existing events.

4.1 Testing Environment

Component	Configuration
API testing tool	Postman
Local Server	XAMPP with Apache
Backend	PHP
Database	MySQL/MariaDB
Database name	event_ticketing_db
API base URL	http://localhost/WebAPIFinalProject/api
Authentication	JWT Bearer Token

4.2 Test Cases and Results

No.	Test case	Method and endpoint	Expected result	Actual result
1	Admin login	POST /auth/login	200; JWT returned	Passed
2	Retrieve all venues	GET /venues	200; venue list returned	Passed
3	Retrieve one venue	GET /venues/1	200; venue details returned	Passed
4	Create venue	POST /venues	201; venue created	Passed
5	Update venue	PATCH /venues/{id}	200; venue updated	Passed
6	Delete test venue	DELETE /venues/{id}	200; venue deleted	Passed
7	Retrieve deleted venue	GET /venues/{id}	404; venue not found	Passed
8	Request without token	GET /venues	401; authentication required	Passed
9	Submit negative capacity	POST /venues	422; validation error	Passed
10	Delete venue linked to events	DELETE /venues/1	409; deletion blocked	Passed
11	Check free time slot	GET /venues/2/availability	200; available is true	Passed

12	Check overlapping time slot	GET /venues/2/availability	200; available is false; conflicting event returned	Passed
13	Check maintenance venue	GET /venues/5/availability	200; available is false	Passed
14	Customer login	POST /auth/login	200; customer JWT returned	Passed
15	Customer attempts to create venue	POST /venues	403; access denied	Passed
16	Submit invalid venue ID	GET /venues/abc	400; invalid ID message	Passed

For availability tests, the date was 14 November 2026. Venue 2 was available between 09:00 and 10:00, while 19:00 to 20:00 overlapped with Muar Music Fest 2026. Venue 5 was unavailable because its status was maintenance.

Error responses were expected outcomes in the negative tests. A test passed when the API returned the appropriate response.

4.3 Testing Evidence

The following Postman screenshots show successful requests and expected error responses. Each screenshot includes the HTTP method, request URL, response status and JSON response.

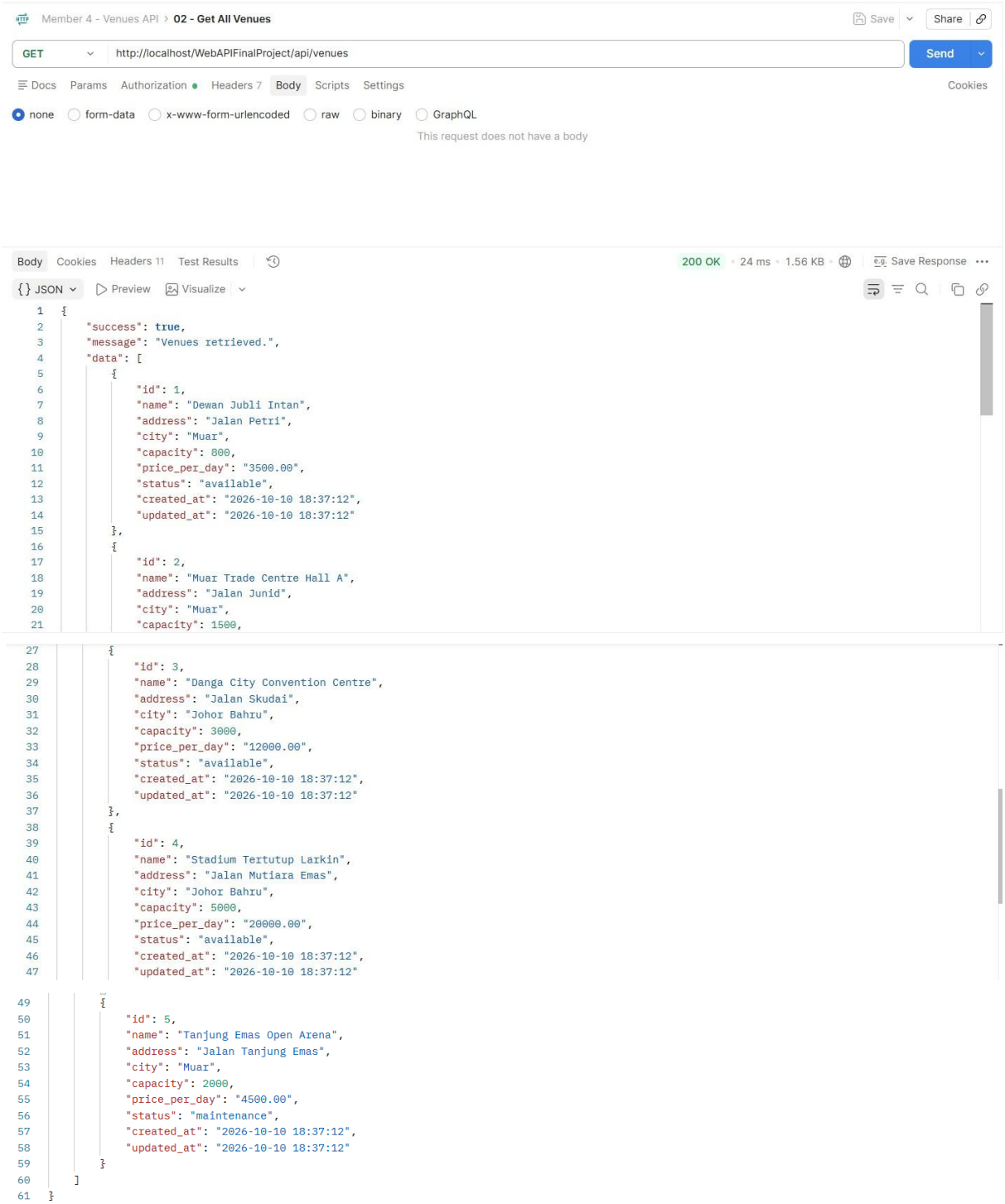


Figure 4.1: Successful retrieval of the venue list (200 OK).

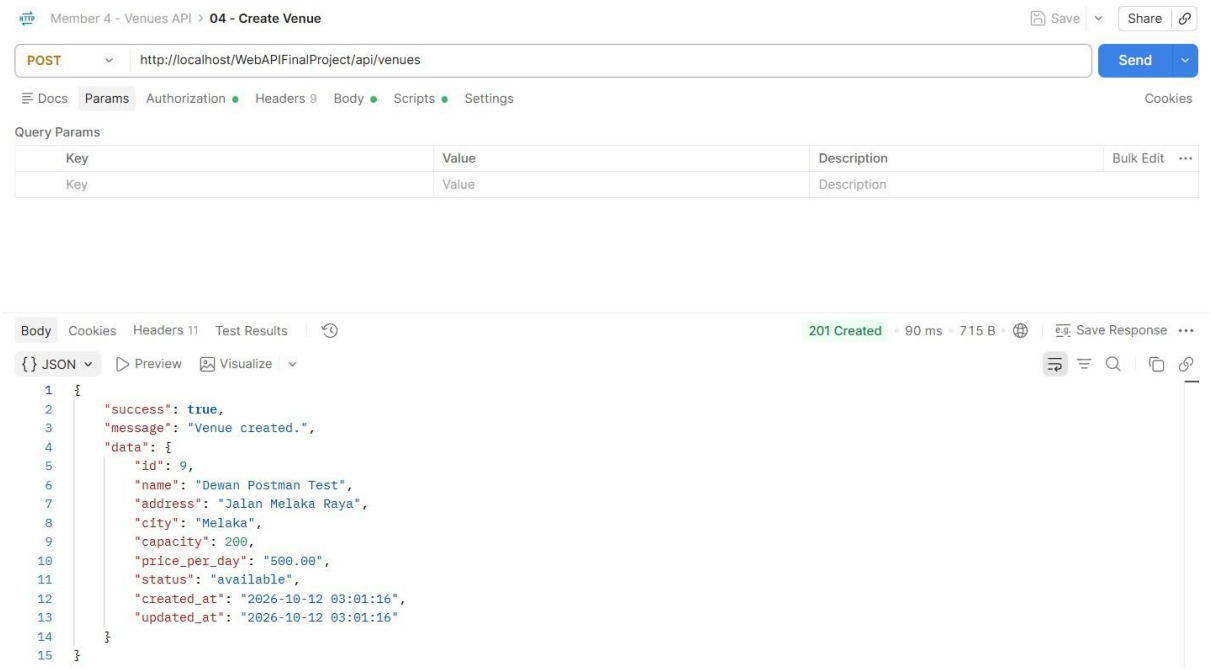


Figure 4.2: Successful creation of a venue (201 Created).

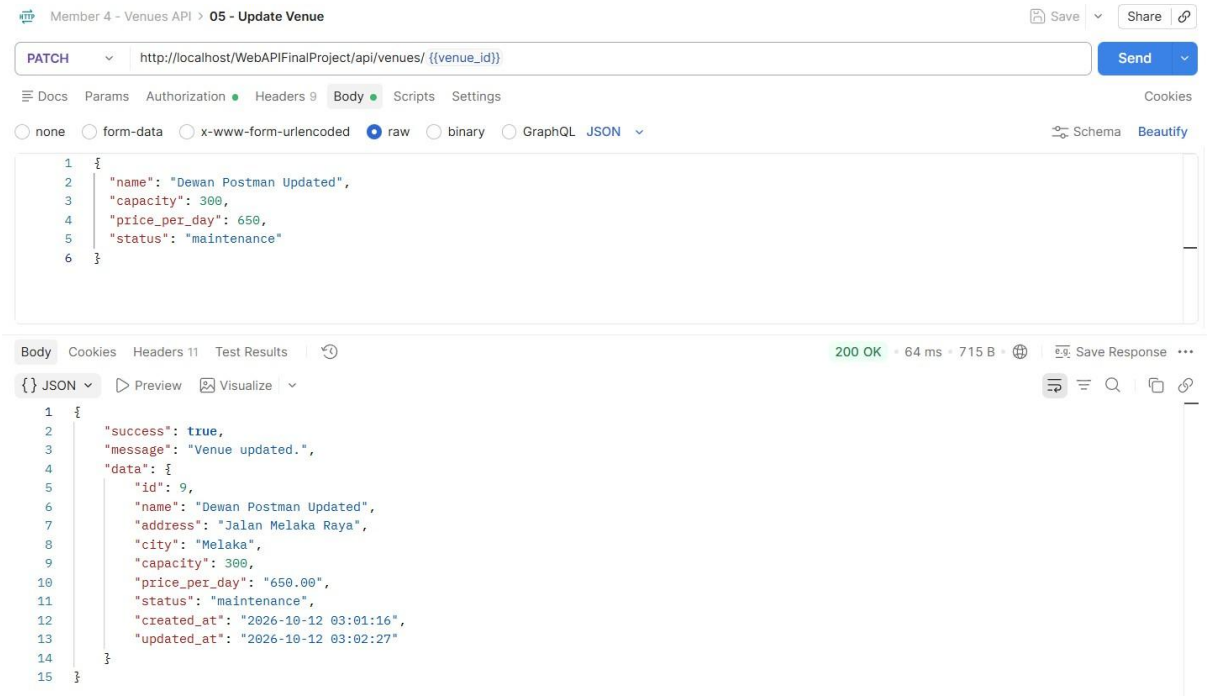


Figure 4.3: Successful update of venue details (200 OK).

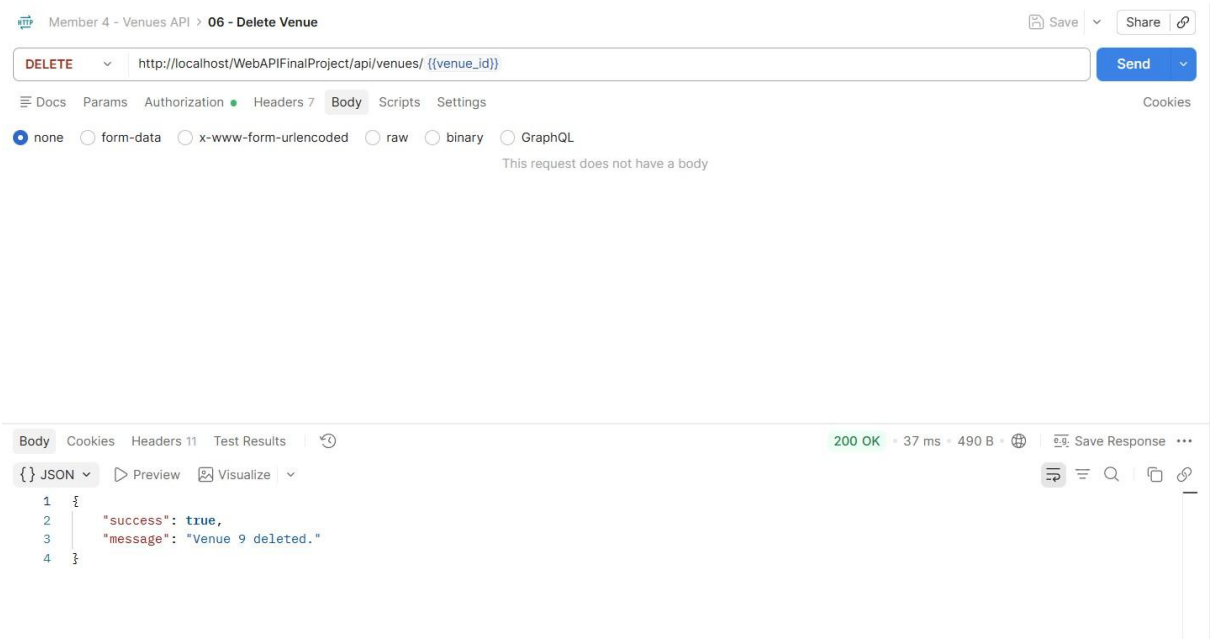


Figure 4.4: Successful deletion of the test venue (200 OK).

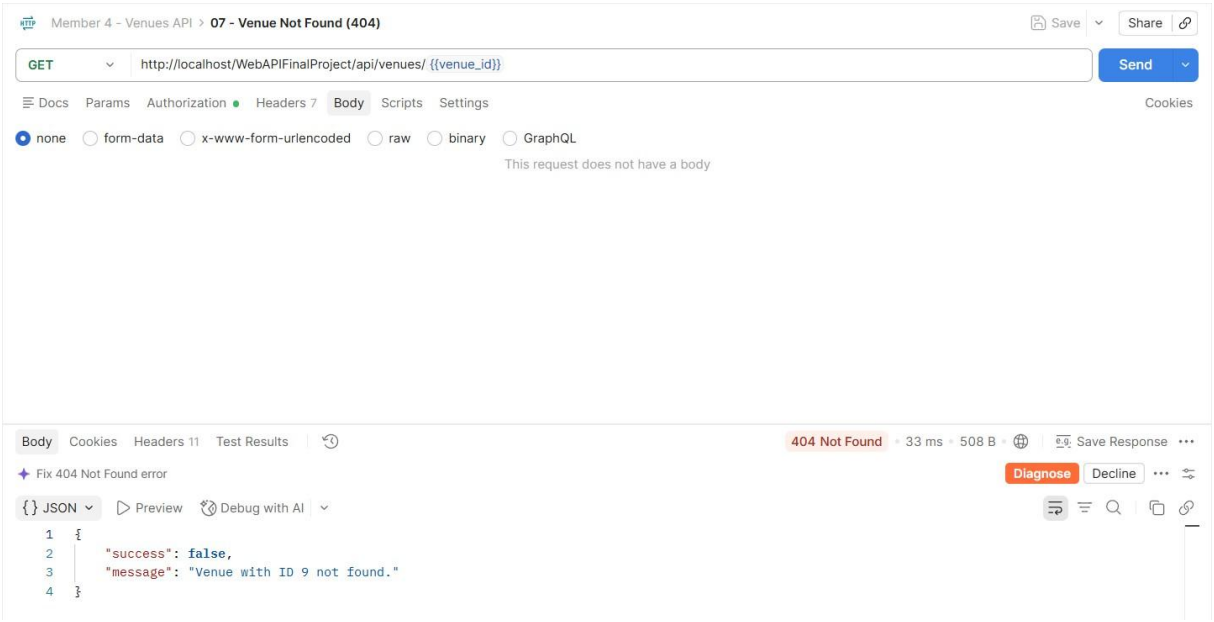


Figure 4.5: Request for a deleted venue returns 404 Not Found.

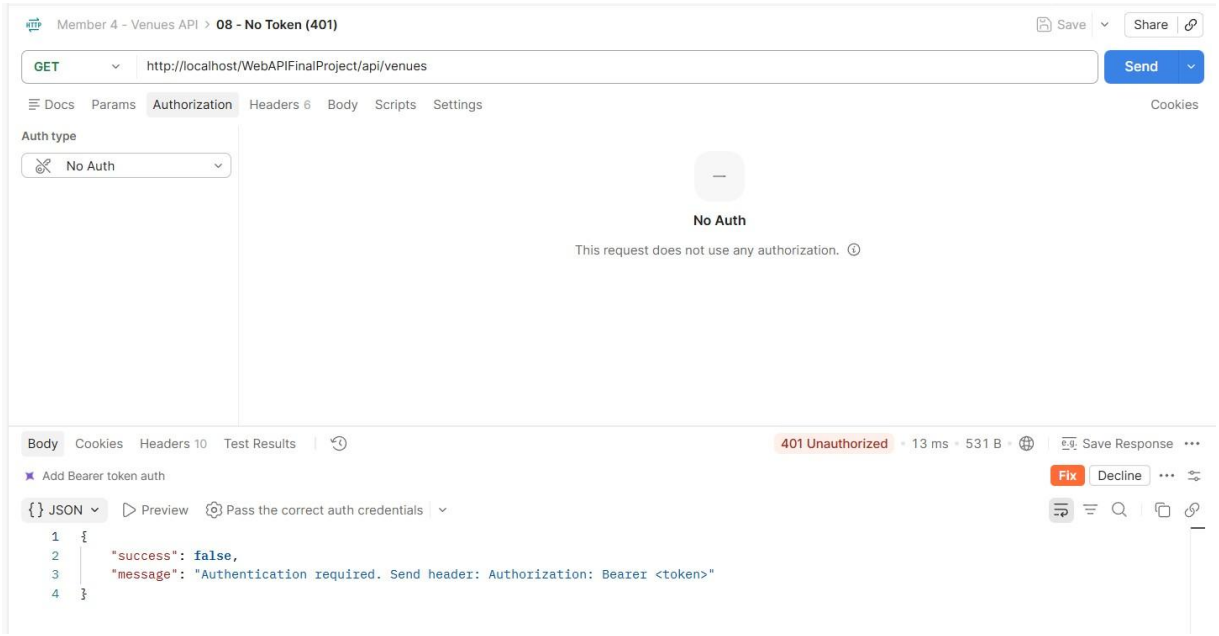


Figure 4.6: A request without a JWT token returns 401 Unauthorized.

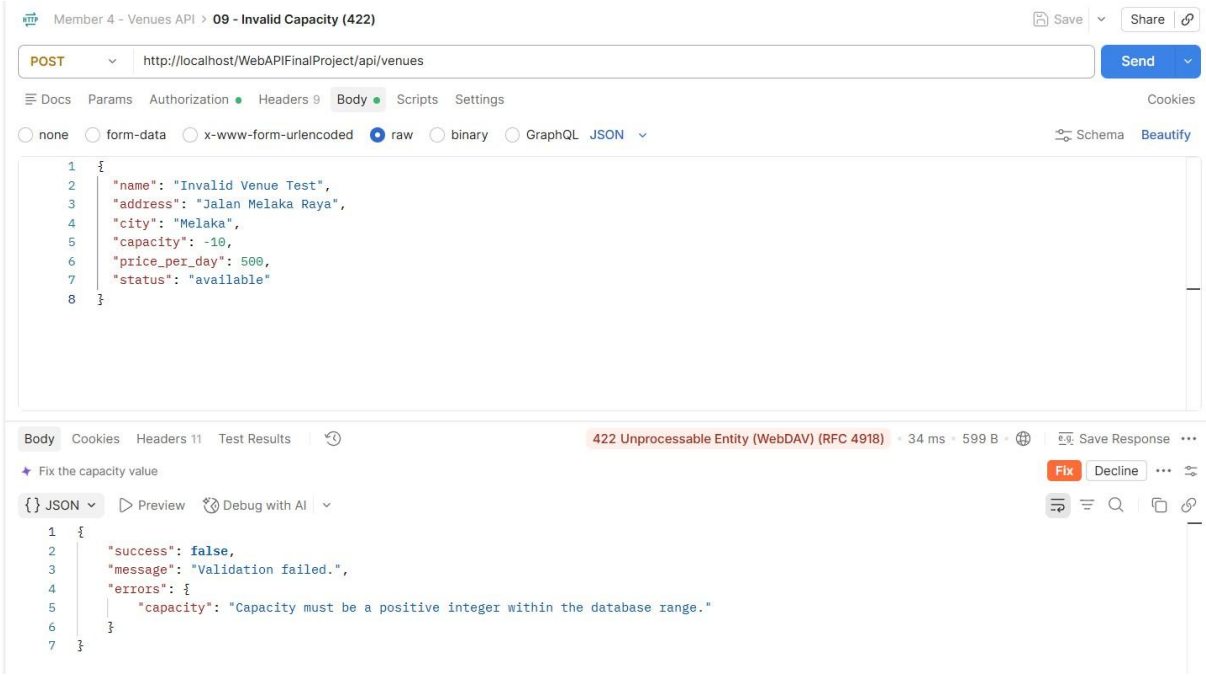


Figure 4.7: A negative venue capacity is rejected with a 422 validation response.

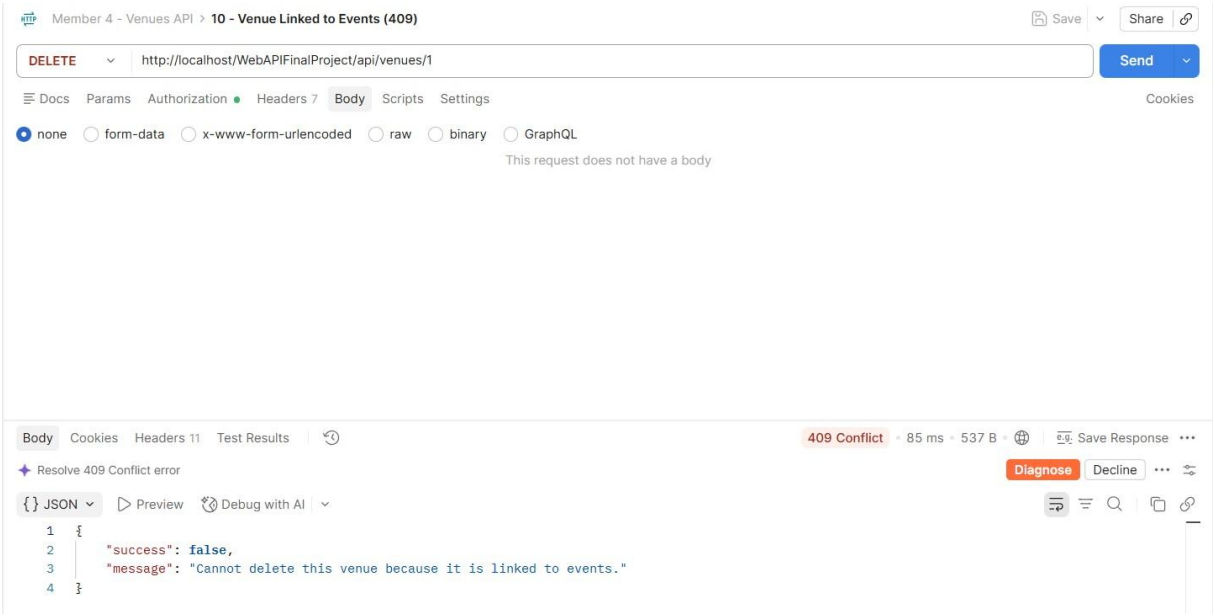


Figure 4.8: Deletion of a venue linked to existing events is blocked with 409 Conflict.

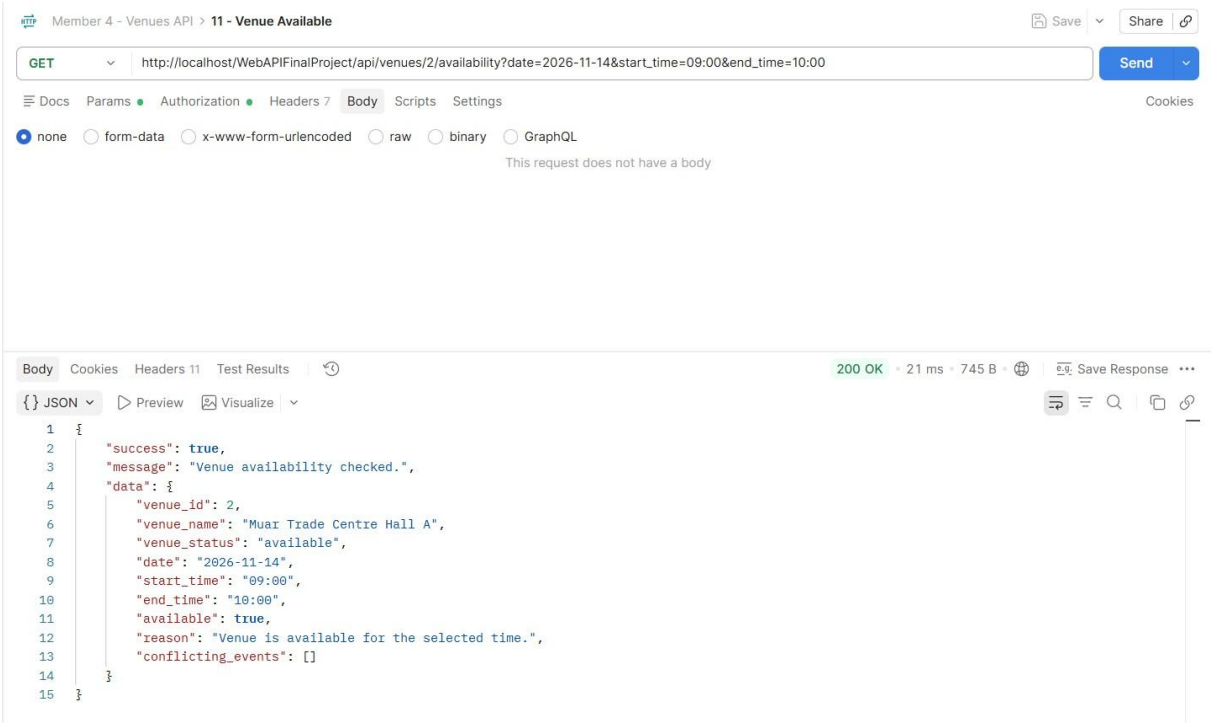


Figure 4.9: Venue 2 is available on 14 November 2026 from 09:00 to 10:00.

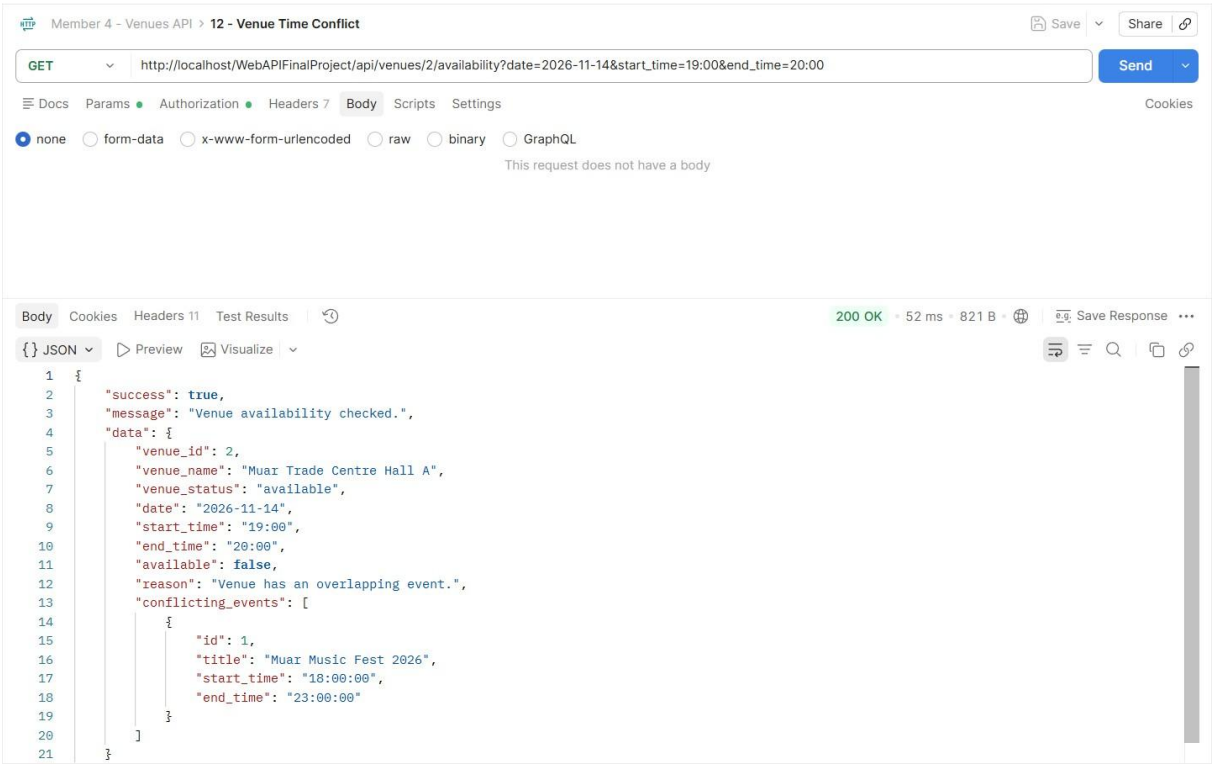


Figure 4.10: Venue 2 is unavailable from 19:00 to 20:00 on 14 November 2026 because the requested slot overlaps with an existing event.

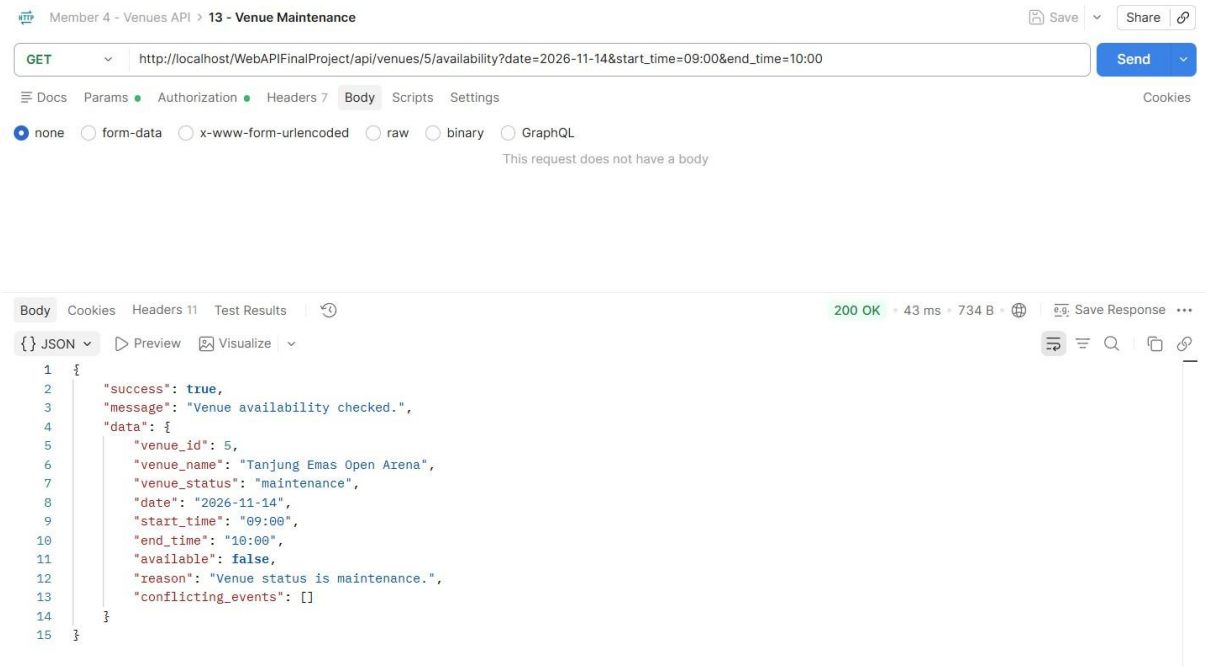


Figure 4.11: Venue 5 is unavailable because its status is maintenance.

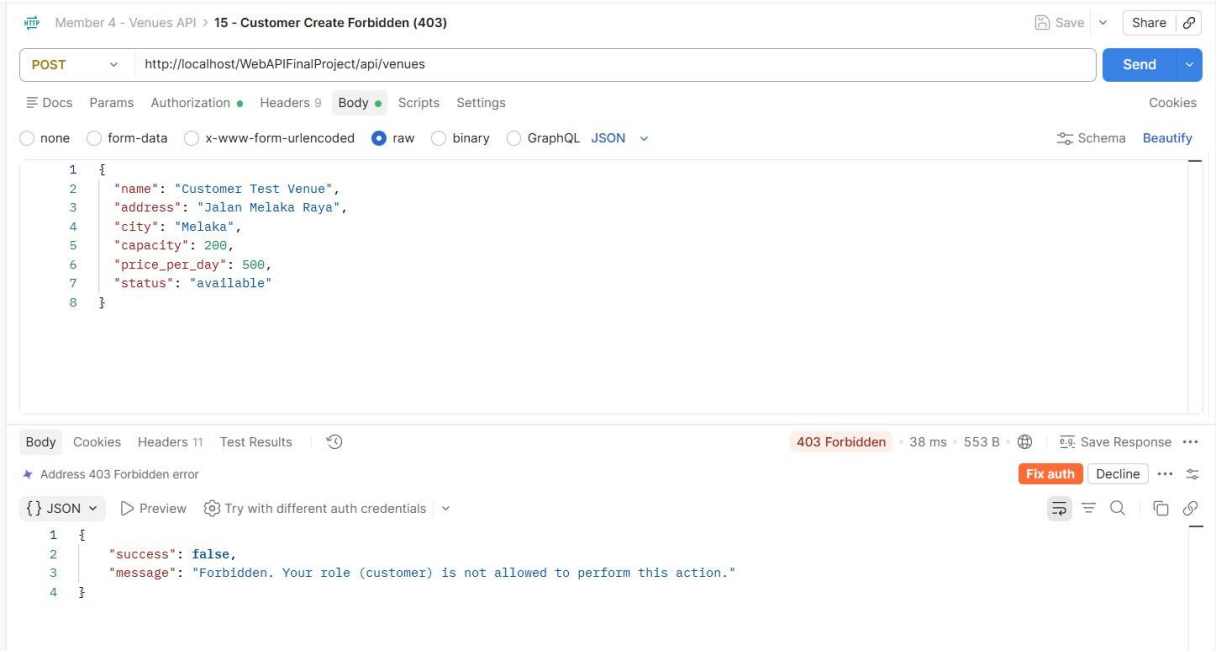


Figure 4.12: A customer is denied permission to create a venue with a 403 Forbidden response.

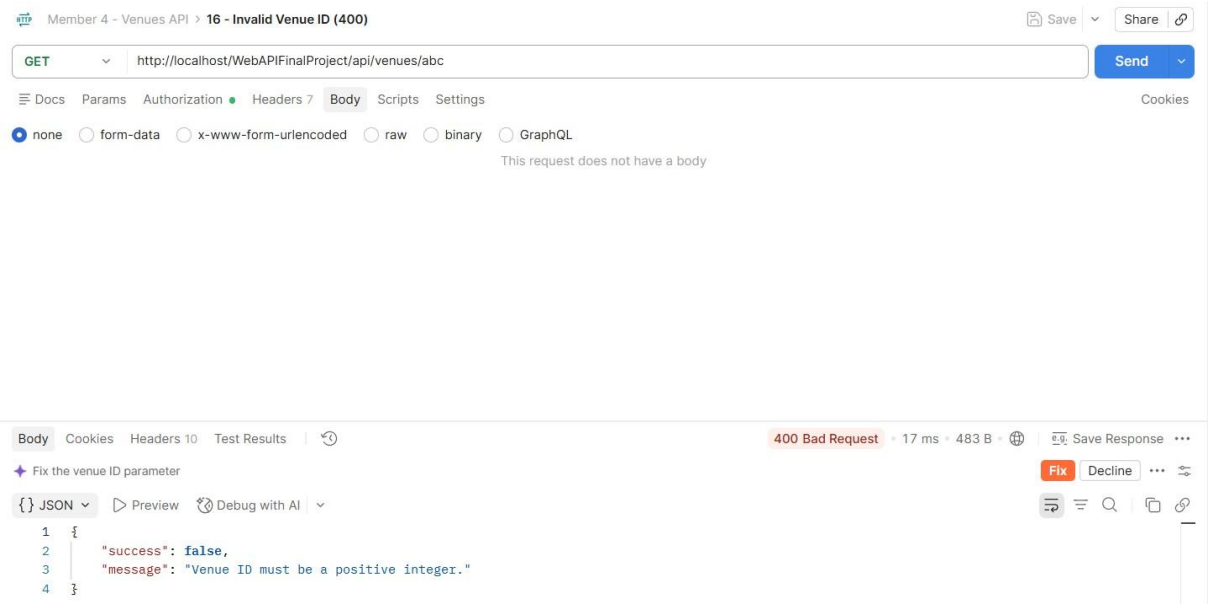


Figure 4.13: A non-numeric venue ID is rejected with 400 Bad Request.

4.4 Exported Postman Collection

The Venues API requests were saved in a Postman collection and exported as Member4_Venues_API.postman_collection.json, stored in the project's docs folder. The collection contains CRUD requests, availability checks and negative test scenarios.

Login scripts automatically populate the admin_token and customer_token variables. The create-venue script stores the new venue ID in venue_id for subsequent update, delete and not-found requests. To repeat the tests, requests should be executed in numerical order. JWT values were cleared before export.

5.0 SYSTEM IMPLEMENTATION

5.1 Shared Backend Implementation

The backend of the Event Ticketing and Management System is constructed as a lightweight, modular RESTful Web API using PHP and MySQL. The architectural pattern separates core system responsibilities into individual layers: routing, database handling, authentication, validation, and endpoint handlers.

1. **Database Connection (`api/config/database.php`)**
   Database interaction utilizes PHP Data Objects (PDO) to ensure secure, parameterized SQL query execution that prevents SQL injection. The database configuration handles dynamic connection instantiation with explicit error handling and UTF-8 encoding.

```
class Database {
    private $host = "localhost";
    private $db_name = "event_ticketing_db";
    private $username = "root";
    private $password = "";
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" . $this-
>db_name,
                $this->username,
                $this->password
            );
            $this->conn->exec("set names utf8");
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Connection error: " . $exception->getMessage();
        }
        return $this->conn;
    }
}
```

2. **Centralised Response Utility (**`api/core/response.php`**)**
   To ensure standard HTTP status codes and uniform JSON outputs across all modules, the system uses a standard response formatting helper class:

```
class Response {
    public static function json($status, $message, $data = null,
$code = 200) {
        http_response_code($code);
        header("Content-Type: application/json; charset=UTF-8");
        header("Access-Control-Allow-Origin: *");
        header("Access-Control-Allow-Headers: Content-Type, Access-
Control-Allow-Headers, Authorization, X-Requested-With");

        $response = [
            "status" => $status,
            "message" => $message
        ];

        if ($data !== null) {
            $response["data"] = $data;
        }

        echo json_encode($response);
        exit();
    }
}
```

3. **JWT Authentication Middleware (**`api/core/auth.php & jwt.php`**)**
   Secured routes such as creating or cancelling bookings require a valid JSON Web Token (JWT) supplied in the Authorization request header (Bearer <token>). The middleware decodes the token, checks signature validity and expiration, and attaches the authenticated user's ID and role (admin or customer) to the request context.

5.2 Frontend Implementation

The frontend architecture follows a componentized, standard client-side model using HTML5, CSS3, JavaScript (Fetch API), and Bootstrap 5 for responsive interface layouts. Each team member implemented a distinct module to fulfill the full system workflow:

Member	Frontend Module	Key Responsibilities & Capabilities
Member 1	Event Browsing Portal	Public event discovery, real-time title search, category filtering, and modal display of event details.
Member 2	Authentication & User Management	Registration, Login modal, JWT local storage persistence, navbar state toggle, and User Profile CRUD.
Member 3	Event Management Dashboard	Admin management portal to create, edit, publish, and archive event records.
Member 4	Venue Management Dashboard	Admin portal to add venues, configure capacities, set pricing, and monitor seat availability.
Member 5	Booking Management & Ticket Verification	Customer ticket booking interface, active booking tracker, cancellation workflow, and third-party QR code generation.

Member 1: Event Browsing & Public Discovery Portal

- **Responsibilities:** Public-facing event directory and search engine.
- **Key Components:**
    - **Interactive Search Bar:** Features client-side live filtering and server-side keyword queries by title, description, or location.
    - **Category Filter Controls:** Navigation chips/dropdowns that dynamically query events by specific categories (e.g., Music, Conference, Sports).
    - **Event Catalog Grid:** Responsive layout displaying event cards with title, cover image, date, venue, ticket price, and remaining seat badge.
    - **Event Details Modal:** Triggered upon selecting an event card to view extended information, venue details, and direct links to reserve tickets.

Member 2: Authentication & User Profile Management

- **Responsibilities:** User onboarding, authentication state, and profile management.
- **Key Components:**
    - **Registration & Login Modals:** Interface forms with client-side field validation (email syntax, password strength).
    - **Session Persistence Manager:** Captures the JWT upon successful login, stores it in browser localStorage, and updates navigation bar state dynamically (e.g., displaying username and Logout button instead of Login/Register).
    - **Customer Profile Dashboard:** Interface allowing users to view registered account info, update profile details (name, phone number, email), and modify their password securely.

Member 3 Implementation Details:

Member 3 implemented the Events API and the event management frontend. The API is part of the shared PHP backend and uses its router, authentication middleware, database connection, and standard JSON response helpers. The frontend sends requests to that API to browse events and to manage them according to the signed-in user's role.

Events API

Main file: api/routes/events.php

- Provides GET /events and GET /events/{id} for published event browsing and event details; supports search, category, venue, status, organiser, and pagination filters.
- Provides POST /events, PUT or PATCH /events/{id}, and DELETE /events/{id} for event management.
- Validates event fields, date and time formats, ticket counts, prices, event status, and venue IDs, returning structured validation errors.
- Uses JWT role checks: admins can manage all events, while organisers can manage only events they own. Guests and customers can view published events.
- Checks venue availability and overlapping event schedules. It prevents deletion when confirmed or linked bookings would be affected.

Event management frontend

Main folder: frontend/member3(syamir)-event-management/

- index.html and style.css provide the event browsing and management interface.
- app.js connects to the shared API, manages the login session, loads event and venue data, applies filters, opens event details, and submits create, edit, and delete requests.
- The interface shows management controls to admin and organiser accounts and hides them from customer accounts. The API URL can be changed from the page's Edit control when the project folder name differs.

For the local demo shown in the browser, set the API URL to match the folder where Apache serves the repository. For example, if the project folder is C:\xampp\htdocs\WebAPIFinalProject-latest, use http://localhost/WebAPIFinalProject-latest/api.

Member 4: Venue Management Dashboard (Admin)

- **Responsibilities:** Management of event venues, capacity allocations, and physical seating configurations.
- **Key Components:**
  - **Venue Directory:** Overview table listing all available halls, auditoriums, and stadiums along with address and capacity specs.
  - **Venue Configuration Form:** Form interface to add new venues with physical seat capacity constraints, address fields, contact details, and base pricing rules.
  - **Availability Tracker:** Dynamic view checking whether a specific venue is booked for selected date/time slots before enabling event assignment.

Member 5 Frontend Details: Booking Management:

The Booking Management interface (`frontend/member5-booking-management/index.html`) is structured into three primary operational blocks:

1. **Event Ticket Reservation Form**: An interactive form that loads available events dynamically, calculates total pricing based on selected ticket quantity, validates venue capacity constraints in real time, and sends reservation requests.
2. **My Bookings Dashboard**: A dynamic list that fetches all bookings tied to the authenticated user. Each booking card displays the event title, date, venue location, ticket count, total price, and status badge (Confirmed vs Cancelled).
3. **QR Code Ticket Modal**: An interactive modal popup triggered from an active booking card. It interfaces with an external QR Code API to generate a scanable ticket containing the unique booking reference, user ID, and event metadata.

5.3 Frontend and API Connection

The client communicates with the REST backend via asynchronous HTTP requests using JavaScript's `fetch()` API.

1. **Booking Creation (`POST /api/routes/bookings.php`)**
   When a user selects an event and submits ticket quantities, the frontend extracts the JWT token from `localStorage` and submits a `POST` request payload.

   Frontend Connection Logic (`app.js`):

```
async function createBooking(eventId, seatsBooked) {
    const token = localStorage.getItem('jwt_token');
    if (!token) {
        alert("Please log in to complete your booking.");
        return;
    }

    try {
        const response = await
fetch('http://localhost/api/routes/bookings.php', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            body: JSON.stringify({
                event_id: eventId,
                seats_booked: seatsBooked
            })
        });

        const result = await response.json();
        if (result.status === 'success') {
            alert("Booking confirmed successfully!");
            loadUserBookings(); // Refresh UI
        } else {
            alert("Booking failed: " + result.message);
        }
    } catch (error) {
        console.error("Error creating booking:", error);
    }
}
```

Backend Endpoint Logic (`api/routes/bookings.php`):

```
if ($method === 'POST') {
    $user_data = Auth::verifyToken();
    $data = json_decode(file_get_contents("php://input"));

    if (empty($data->event_id) || empty($data->seats_booked)) {
        Response::json("error", "Incomplete booking payload.", null,
400);
    }
    $stmt = $db->prepare("SELECT available_seats, ticket_price FROM
events WHERE id = :event_id");
    $stmt->bindParam(":event_id", $data->event_id);
    $stmt->execute();
    $event = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$event || $event['available_seats'] < $data->seats_booked)
{
        Response::json("error", "Insufficient seats available.",
null, 400);
    }

    $total_price = $event['ticket_price'] * $data->seats_booked;
    $booking_reference = "BK-" . strtoupper(uniqid());
    $db->beginTransaction();

    $bookingQuery = "INSERT INTO bookings (booking_reference,
user_id, event_id, seats_booked, total_price, status)
                     VALUES (:ref, :user_id, :event_id, :seats,
:price, 'Confirmed')";
    $bStmt = $db->prepare($bookingQuery);
    $bStmt->execute([
        ':ref' => $booking_reference,
        ':user_id' => $user_data->id,
        ':event_id' => $data->event_id,
        ':seats' => $data->seats_booked,
        ':price' => $total_price
    ]);

    $updateQuery = "UPDATE events SET available_seats =
available_seats - :seats WHERE id = :event_id";
    $uStmt = $db->prepare($updateQuery);
    $uStmt->execute([':seats' => $data->seats_booked, ':event_id' =>
$data->event_id]);

    $db->commit();
```

```
    Response::json("success", "Booking created successfully.", [
        "booking_reference" => $booking_reference,
        "total_price" => $total_price
    ], 201);
}
```

2. **Booking Cancellation Workflow (`PUT /api/routes/bookings.php`)**
   Users can cancel an existing active booking. The system updates the booking status to Cancelled and automatically increments the event's available seat count by the cancelled number of tickets.

   Frontend Cancellation Logic:

```
async function cancelBooking(bookingId) {
    if (!confirm("Are you sure you want to cancel this booking?"))
return;

    const token = localStorage.getItem('jwt_token');

    const response = await
fetch('http://localhost/api/routes/bookings.php', {
        method: 'PUT',
        headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${token}`
        },
        body: JSON.stringify({
            booking_id: bookingId,
            status: 'Cancelled'
        })
    });

    const result = await response.json();
    if (result.status === 'success') {
        loadUserBookings();
    }
}
```

3. **Third-Party Integration: External QR Code Generator API**
   To provide valid digital tickets, Member 5 integrated an external third-party QR Code Generator API (goqr.me API engine via [https://api.qrserver.com/v1/create-qr-code/]). When a user clicks "View Ticket / QR", the frontend constructs an encoded payload combining Booking Reference, User ID, Event Name, and Seat Count, then requests a high-resolution QR code image URL dynamically.

```
function generateTicketQR(bookingRef, eventTitle, seats) {
    const qrData = encodeURIComponent(
        JSON.stringify({
            ref: bookingRef,
            title: eventTitle,
            seats: seats,
            verified_by: "EventHub Platform"
        })
    );
    const qrApiUrl = `https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=${qrData}`;

    document.getElementById('ticketQRImage').src = qrApiUrl;
    document.getElementById('ticketModalRef').innerText =
bookingRef;

    const ticketModal = new
bootstrap.Modal(document.getElementById('ticketModal'));
    ticketModal.show();
}
```

6.0 SYSTEM DEMONSTRATION

6.1 Member 1: Event Browsing and Public Discovery Portal

Member 1 was responsible for developing the front-end event discovery portal, enabling unregistered visitors and logged-in users to search, filter, and inspect available events across the platform.

Interface 1: Event Directory & Catalog Landing Page

The primary landing view loads all active events dynamically from the database. Events are displayed in a responsive grid layout using Bootstrap cards, featuring cover images, title, date, venue location, ticket price, and dynamic seat badges (Available vs Sold Out).

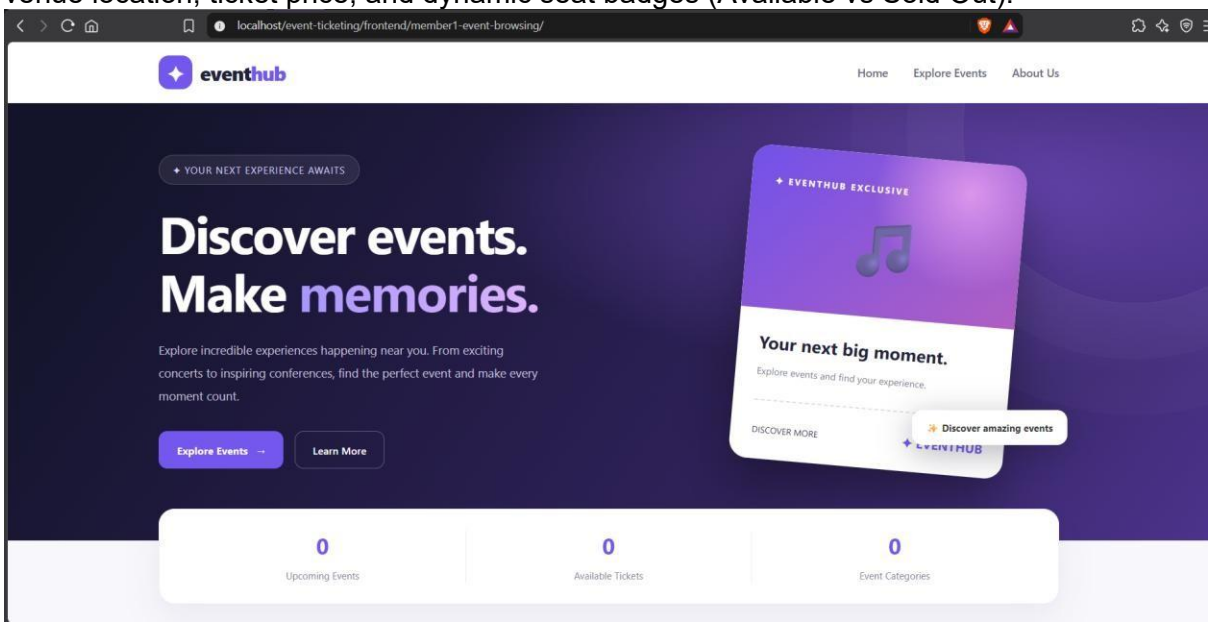


Figure 6.1.1 Event Directory Grid View

Interface 2: Live Keyword Search & Category Filtering

Users can search for specific events by entering titles, descriptions, or locations into the live search input bar. Selecting category navigation buttons (e.g., Music, Tech Conferences, Sports) dynamically filters the displayed cards without requiring a full page reload.

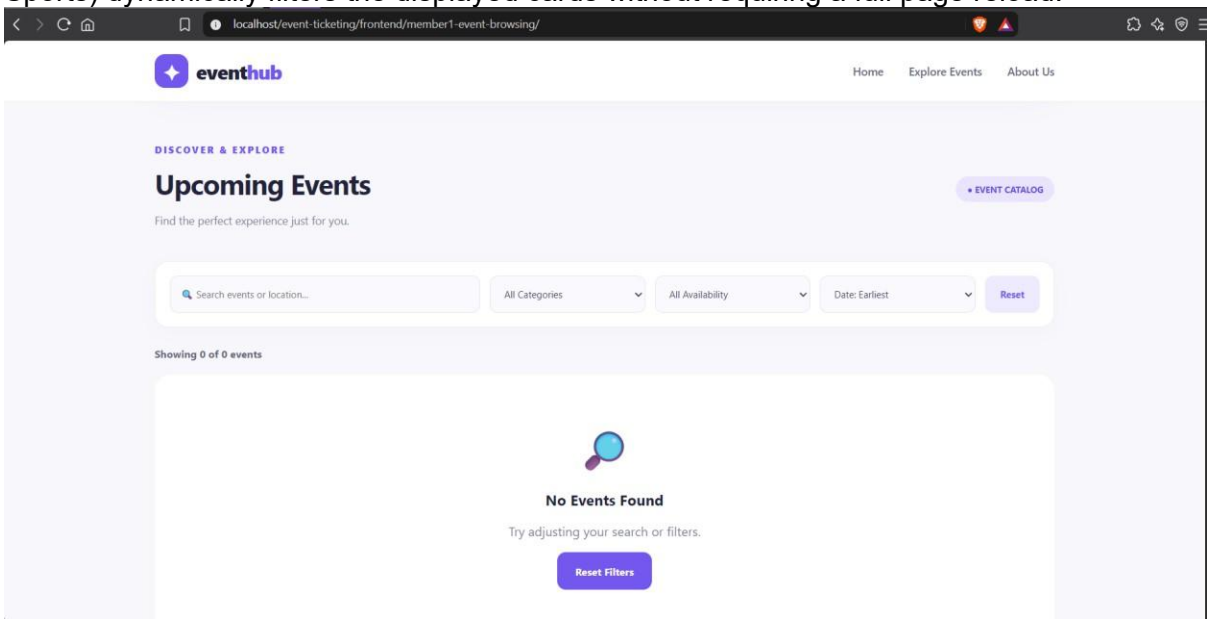


Figure 6.1.2 Dynamic Search and Filter Response

6.2 Member 2: Authentication and User Profile Management

Member 2 developed the user management ecosystem, including user registration, secure authentication, local session persistence via JWT, dynamic navigation bar state updates, and account profile management.

Interface 1: User Registration Modal & Input Validation

New users register by completing a modal form requesting full name, email address, phone number, and password. Client-side script validates email syntax and password strength before submitting data to the server.

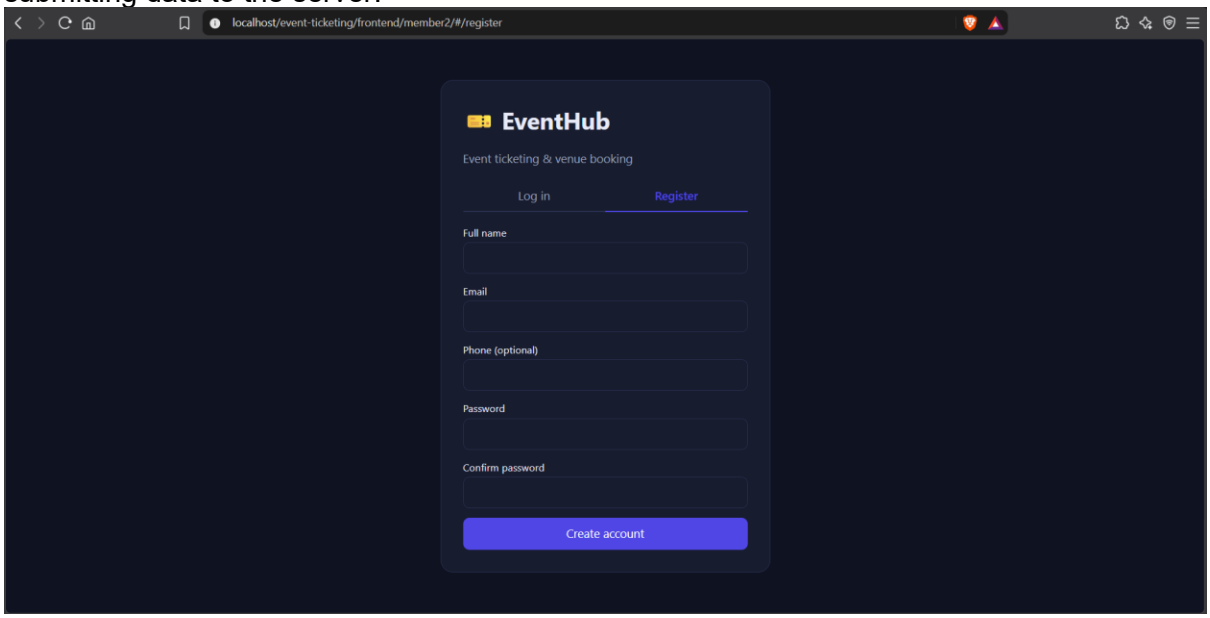


Figure 6.2.1 User Registration Form with Validation

Interface 2: User Profile Management Dashboard

Authenticated users can access their profile dashboard to inspect registered account details, update personal contact information, or change their account password.

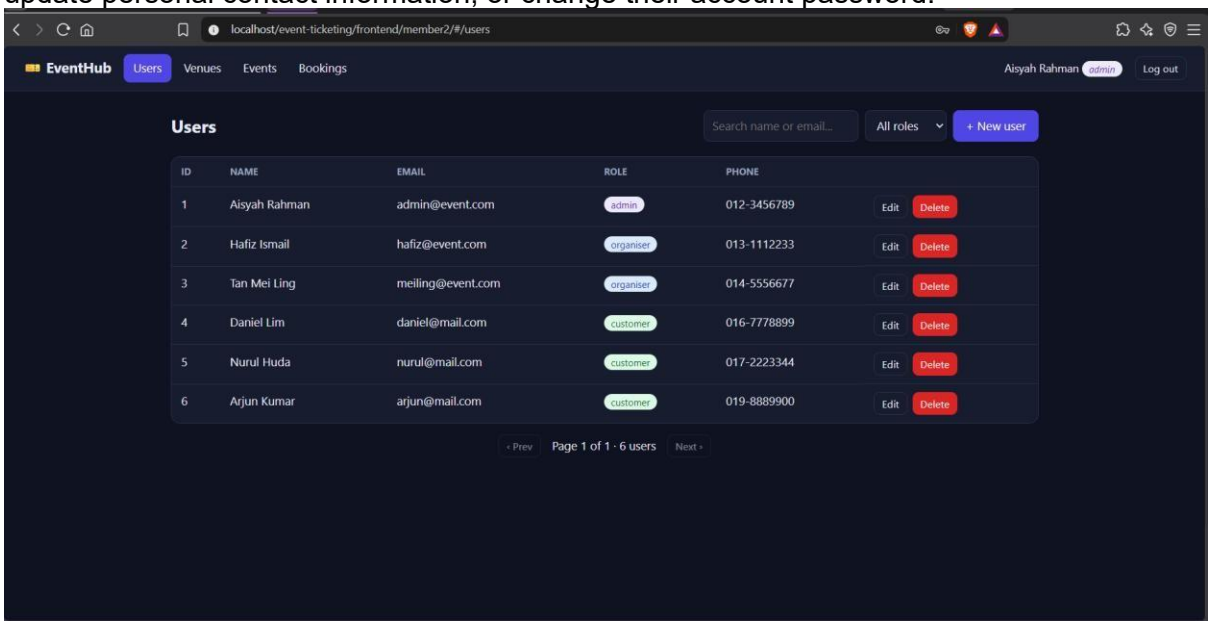


Figure 6.2.2 User Profile CRUD Interface

6.3 Member 3: Event Management Dashboard

Screenshot 1 | Guest event list with a filter applied

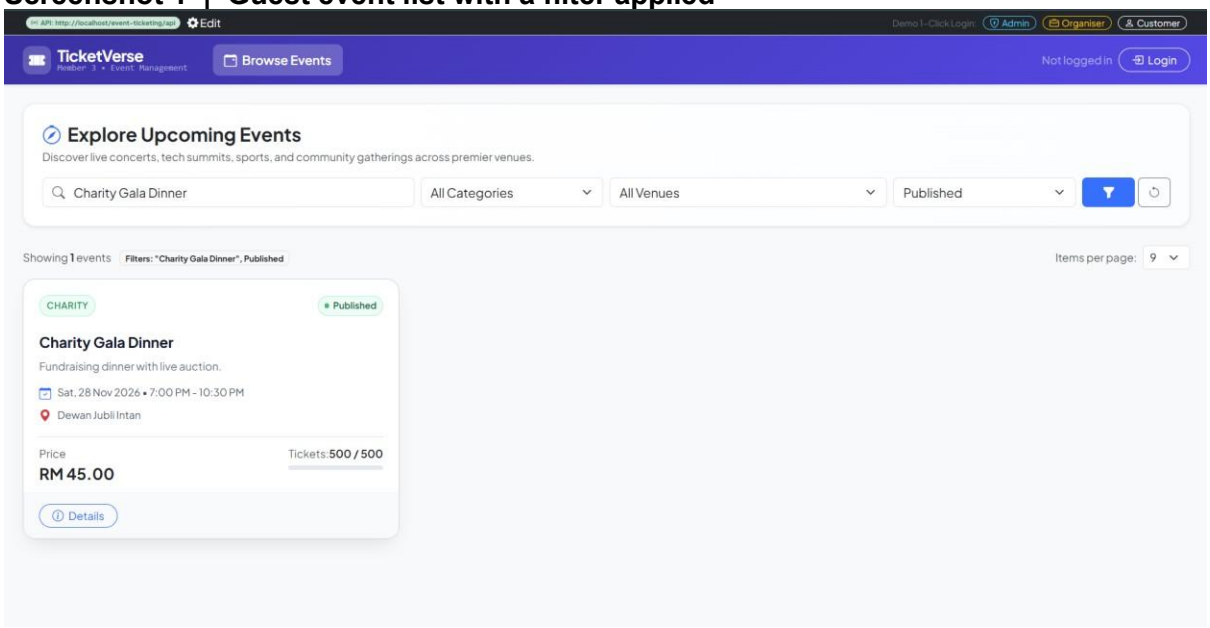


Figure 6.3.1 Guest Browsing with a search filter

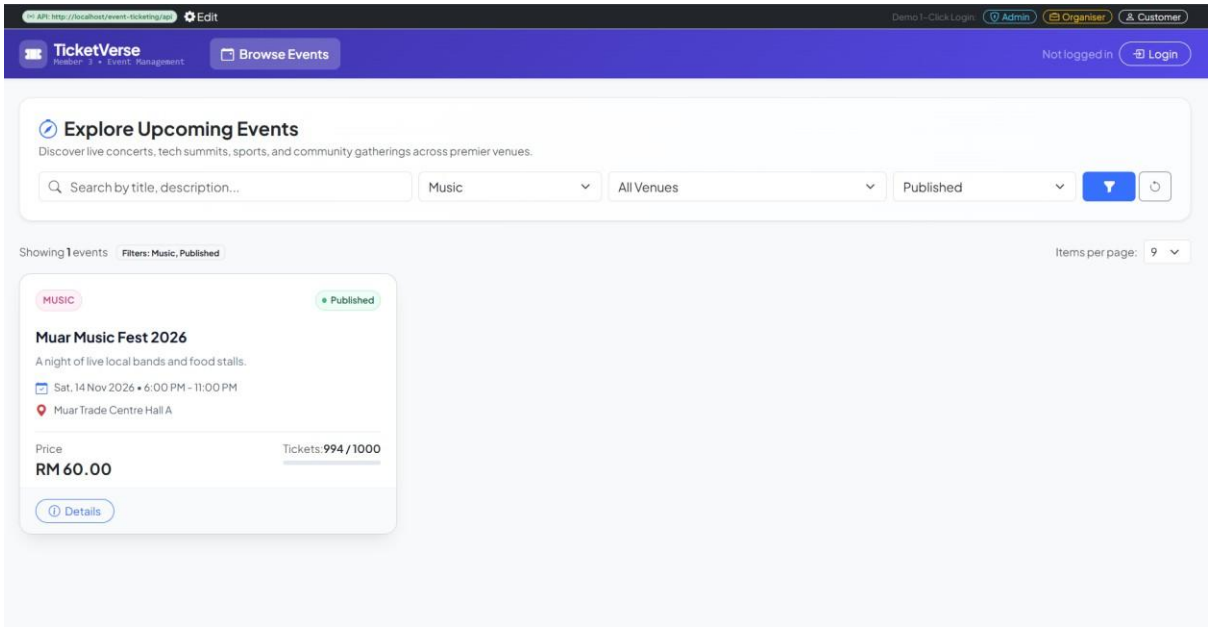


Figure 6.3.2 Guest Browsing with a Category Filter

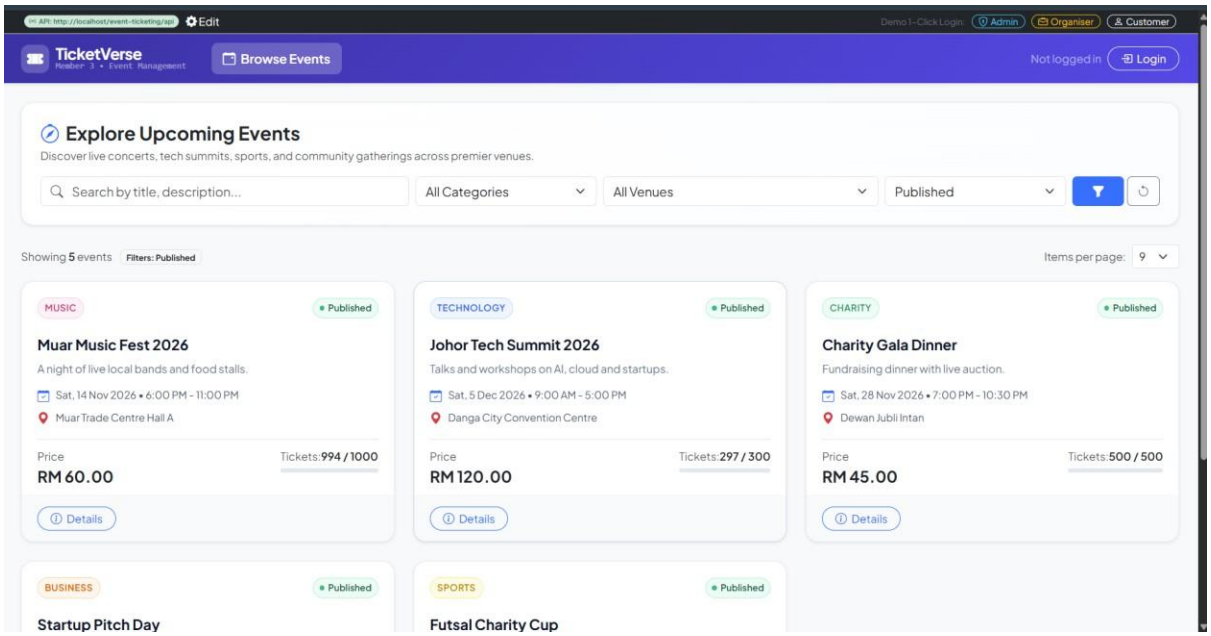


Figure 6.3.3 Guest Browsing with a Status Filter

Screenshot 2 | Event details view

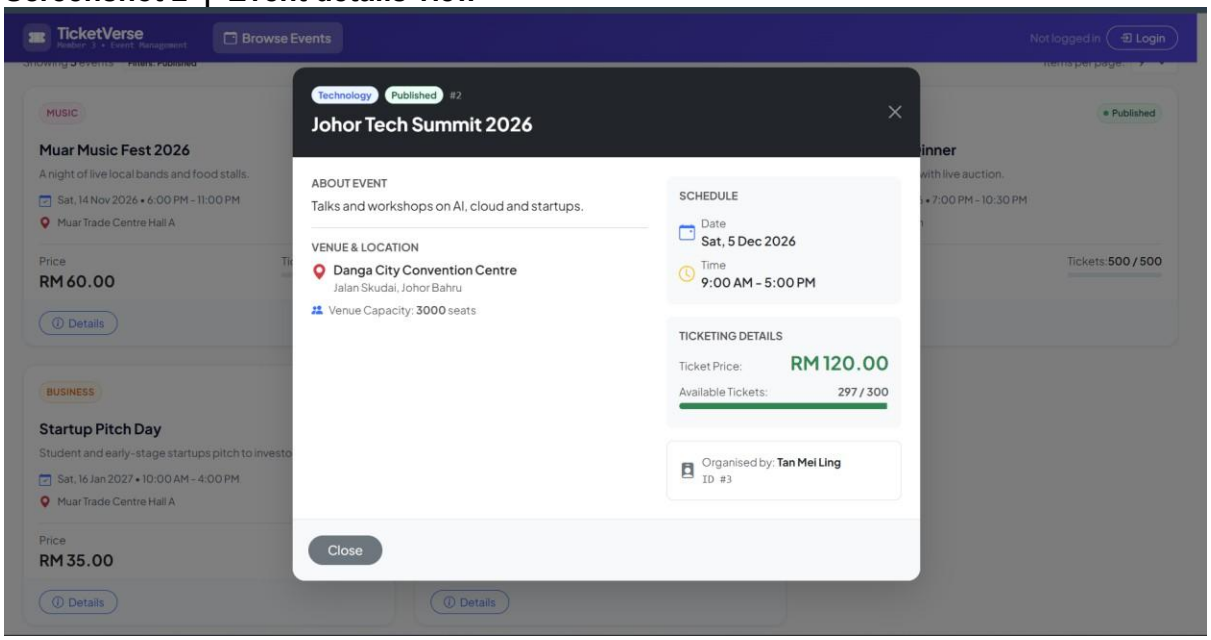


Figure 6.3.4 Organizer Event Details View

Screenshot 3 | Organiser event management page

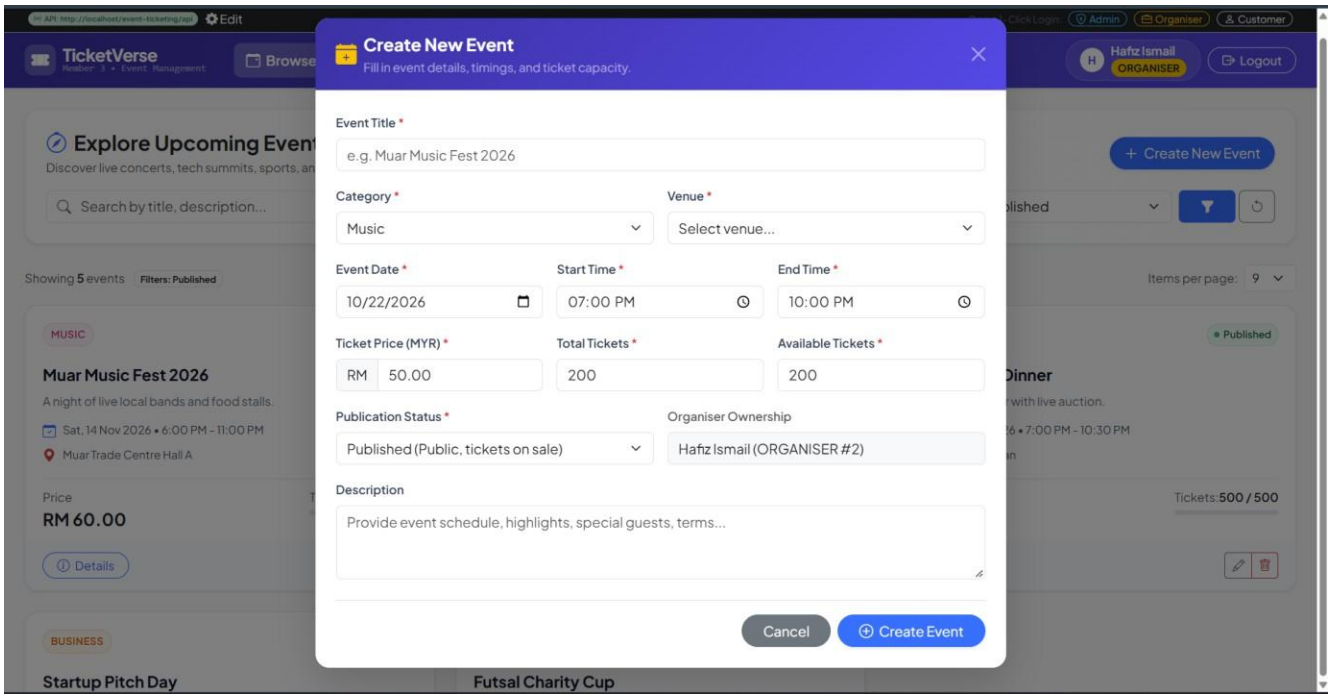


Figure 6.3.5: Organizer event management view

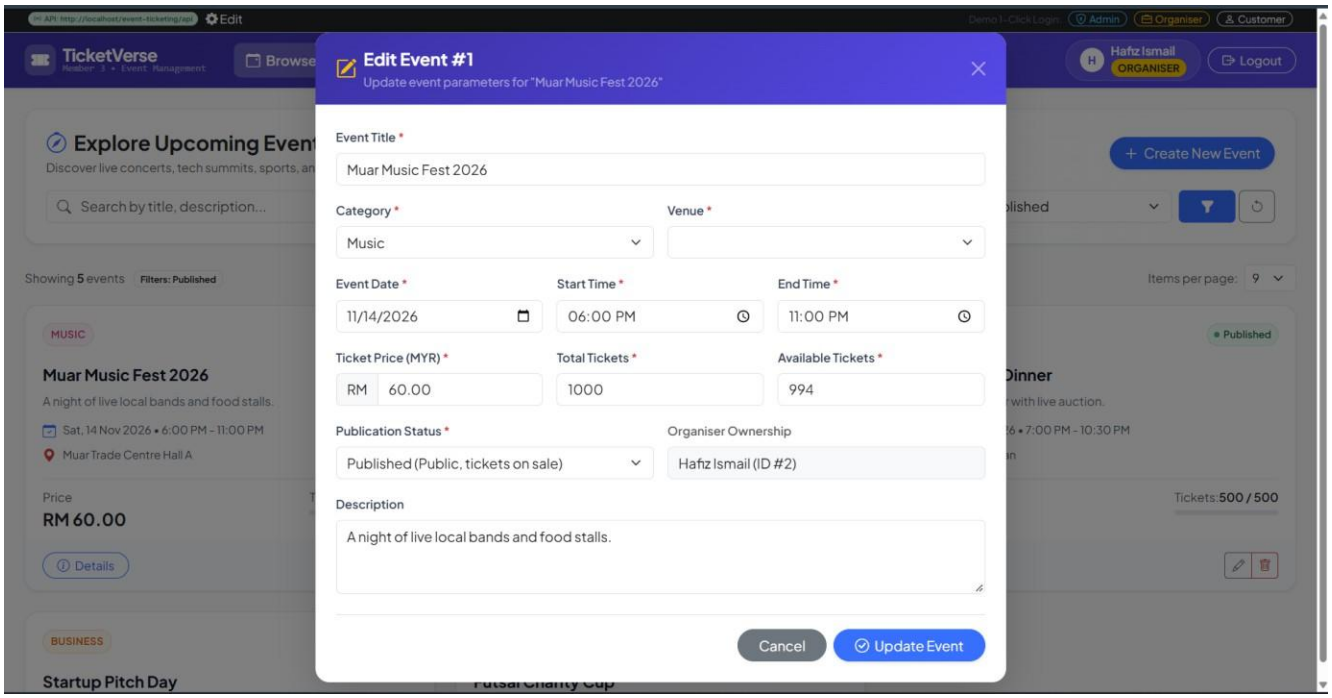


Figure 6.3.6: Organizer Editing event view

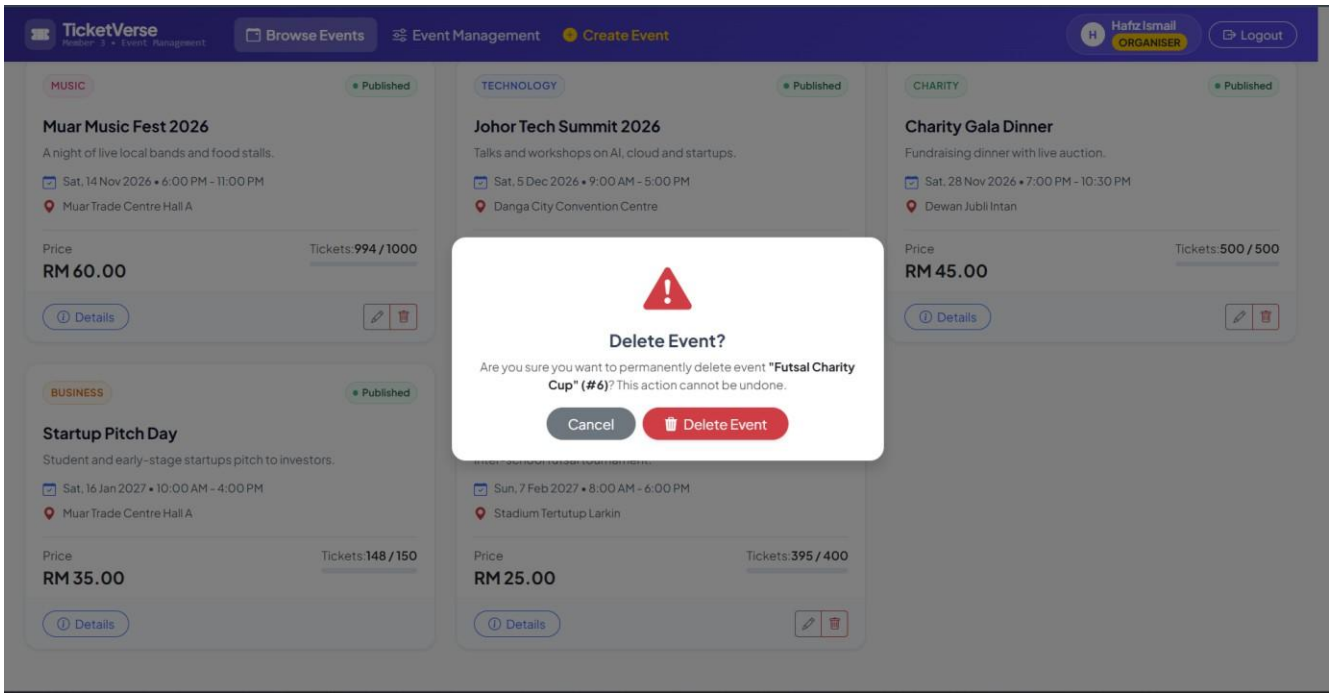


Figure 6.3.7: Organizer confirmation to delete an event

Screenshot 4 | Create or edit success state

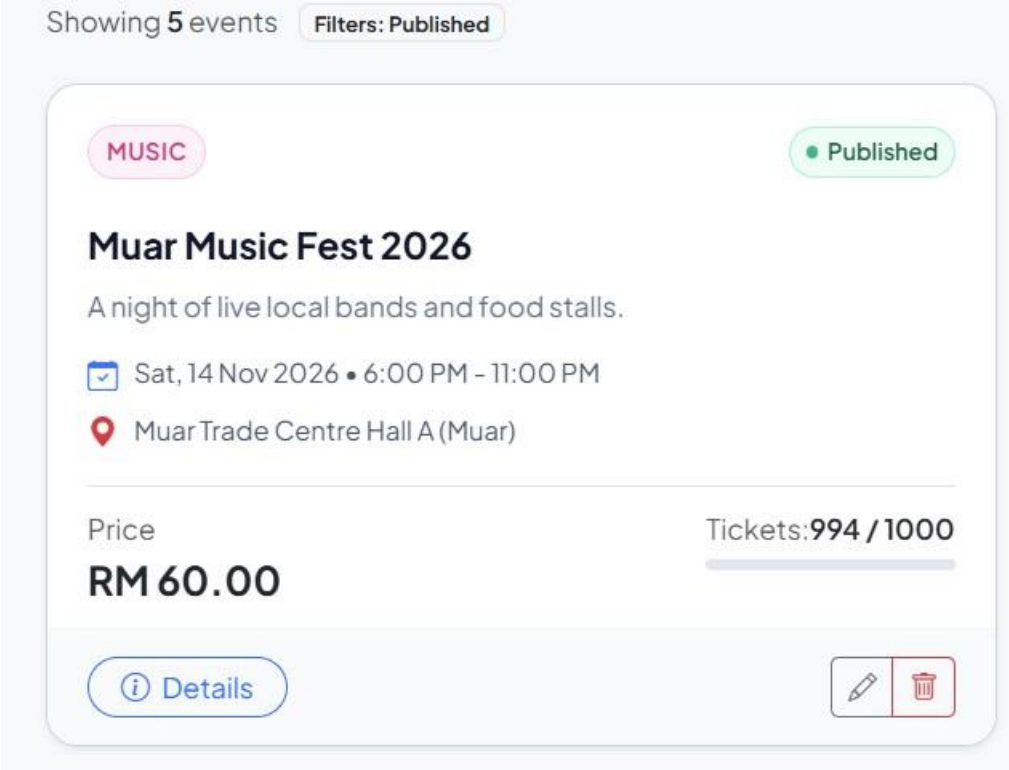


Figure 6.3.8: Before editing an event details

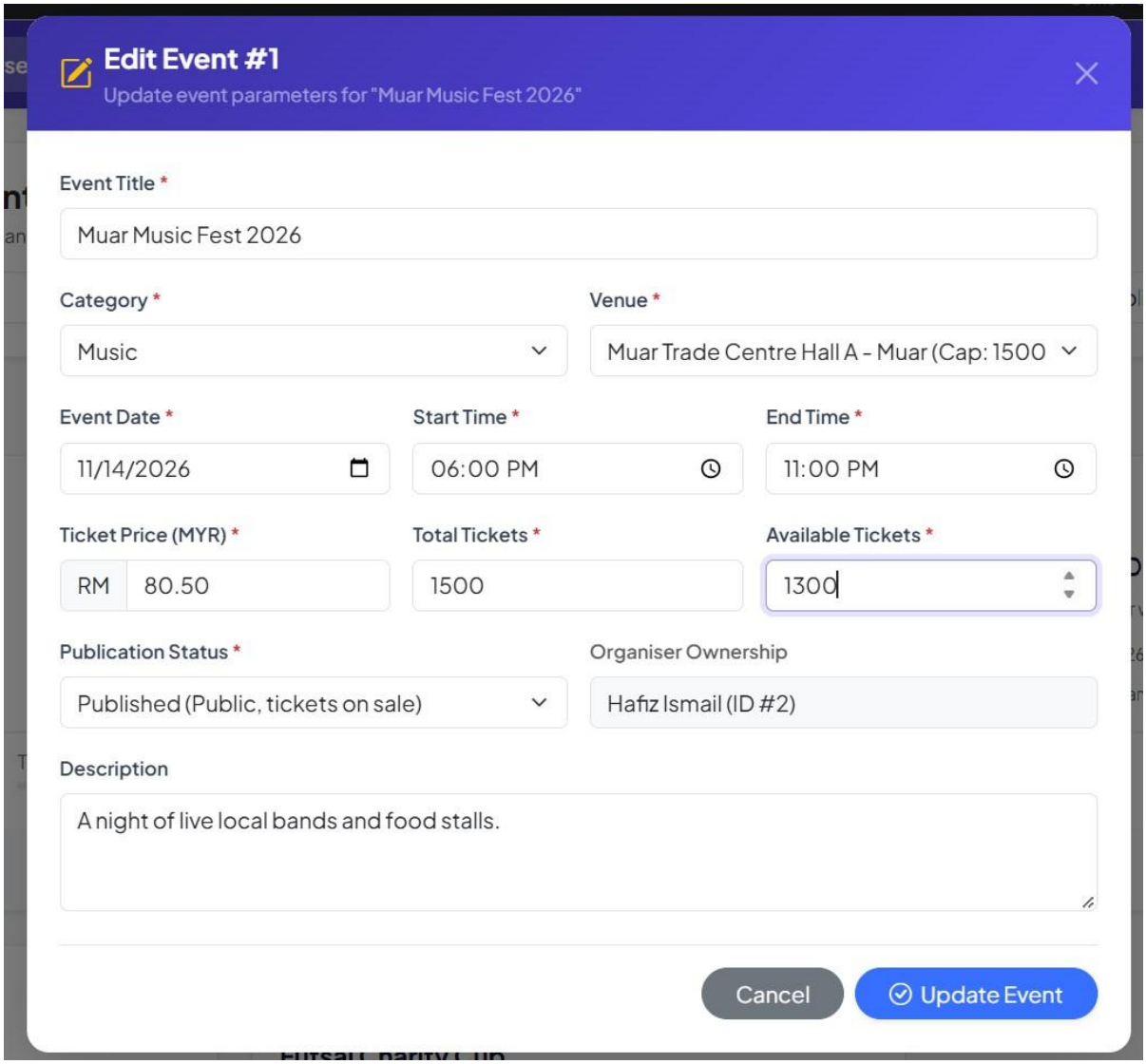


Figure 6.3.9: Editing an event view

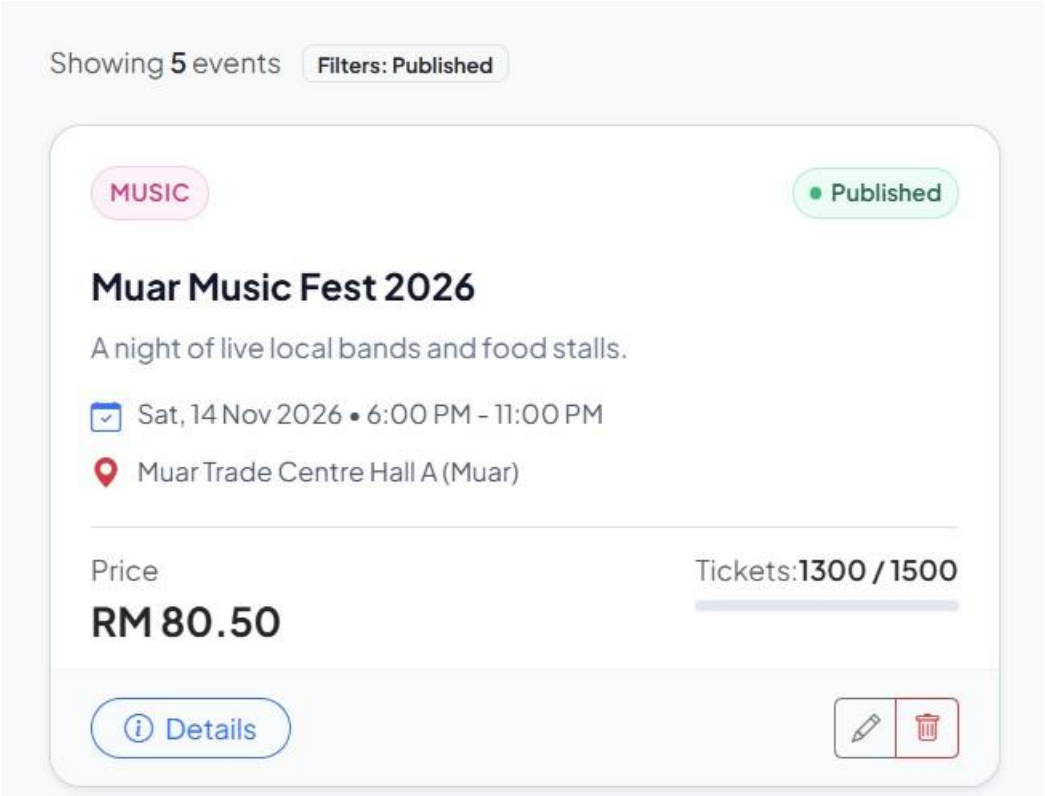


Figure 6.3.10: Result after editing an event

6.4 Member 4: Venue Management Dashboard

Member 4 developed administrative interfaces for configuring physical event venues, setting seat capacity constraints, managing facility details, and preventing venue scheduling overlaps.

Interface 1: Venue Directory Overview

An administrative table lists all system venues, displaying information such as venue name, address, max seating capacity, base rental price, and current availability status.

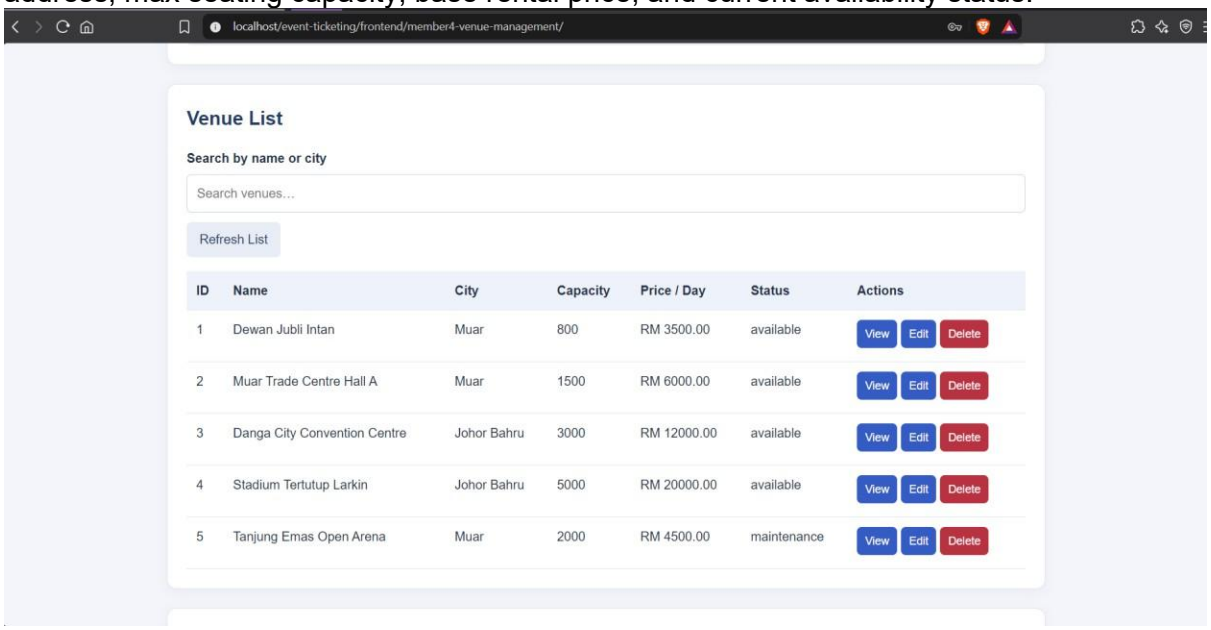


Figure 6.4.1 Venue Management Table

Interface 2: Venue Creation & Configuration Form

Admins can add new physical spaces by specifying the facility name, full street address, contact phone, total seating capacity, and standard booking price.

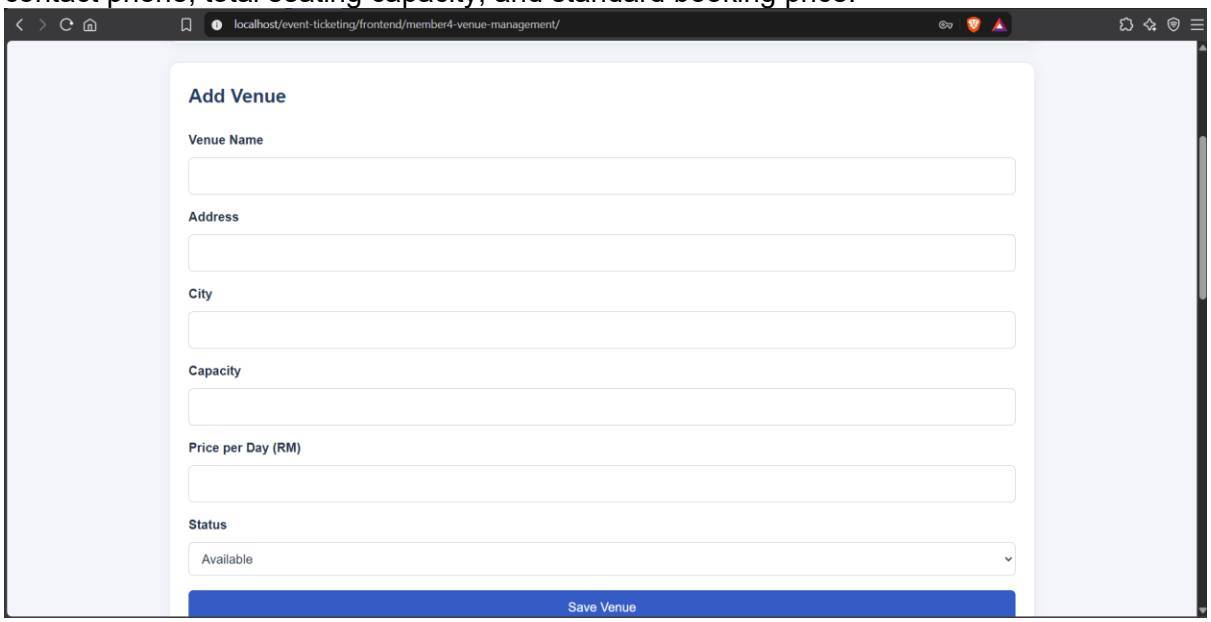


Figure 6.4.2 Add Venue Configuration Form

Interface 3: Venue Availability and Scheduling Tracker

To avoid double-booking issues, this view allows administrators to inspect venue usage schedules across specific dates before assigning an event to a location.

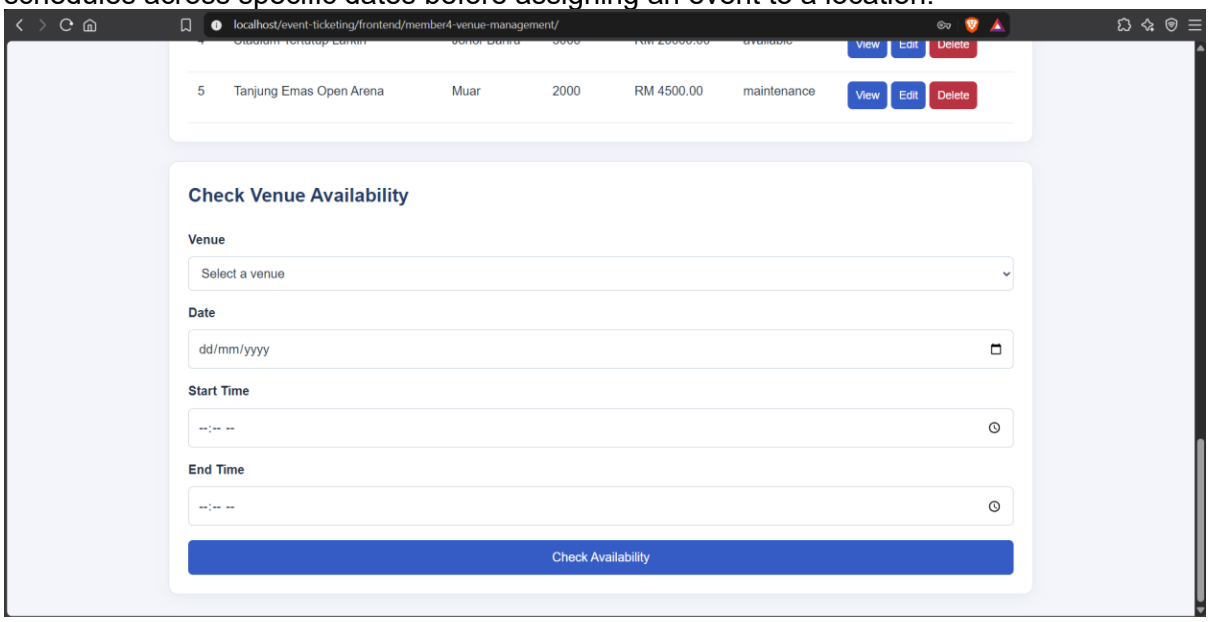


Figure 6.4.3 Venue Availability Checker Interface

6.5 Member 5: Booking Management

Member 5 implemented customer ticket reservation processing, real-time total price calculation, user booking tracking, cancellation workflows with automated seat restoration, and dynamic QR code ticket generation.

Interface 1: Booking New Ticket

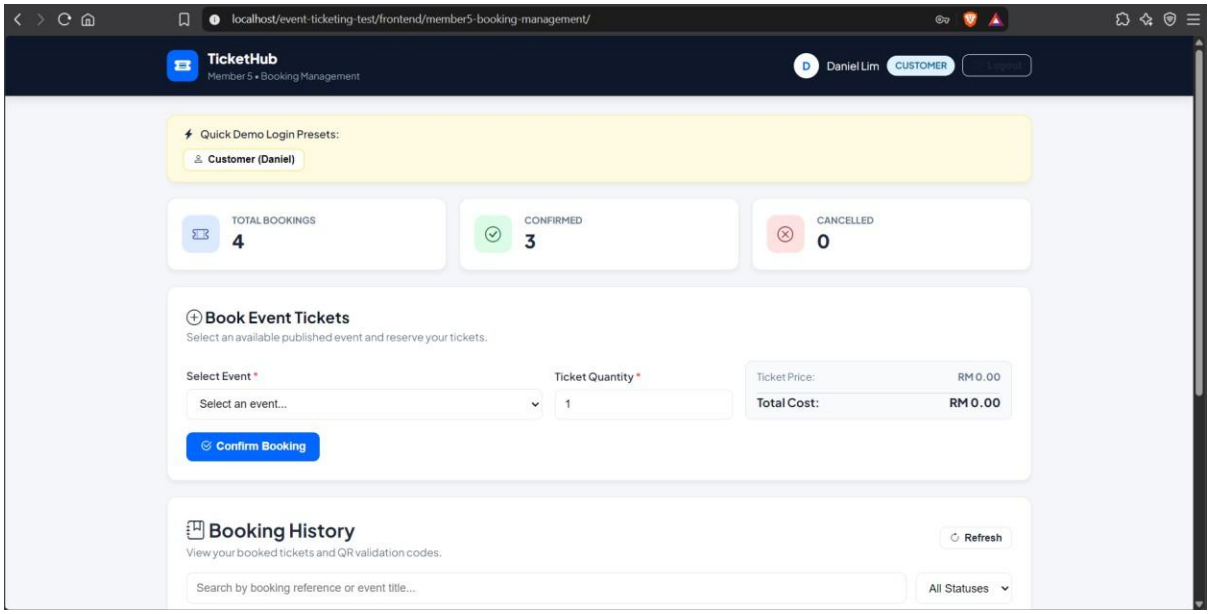


Figure 6.5.1 Ticket Booking

Interface 2: "My Bookings" Customer Dashboard

Customers can track all their ticket reservations on a dedicated dashboard. Each card presents the unique booking reference code, event title, date/time, seat count, total paid amount, and status badge (Confirmed vs Cancelled).

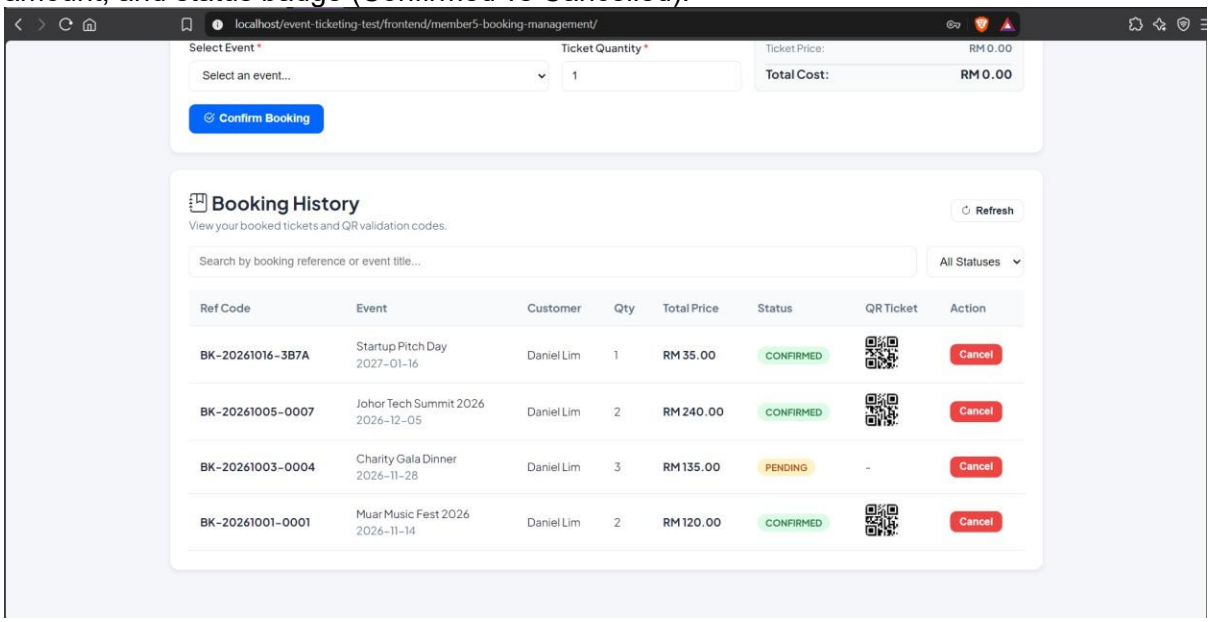


Figure 6.5.2 Customer "My Bookings" Dashboard View

7.0 Project Repository

7.1 Repository Information

- Platform: GitHub
- Repository name: event-ticketing
- Repository URL: https://github.com/syakirin1908/WebAPIFinalProject.git
- Visibility: Private, with the lecturer or tutor added as a collaborator (or public if the course requires it).

7.2 Repository Structure

```
event-ticketing/
├── README.md            setup + API guide
├── .gitignore
├── database/            schema.sql, seed.sql
├── api/                 shared backend (PHP + MySQL)
│   ├── index.php        router
│   ├── config/          config.php, database.php
│   ├── core/            jwt.php, auth.php, response.php, validator.php
│   └── routes/          auth.php, users.php (+ events, venues, bookings)
├── frontend/            each member's own frontend (member1 ... member5)
├── postman/             collection + screenshots
└── docs/                report, ERD.png, contribution evidence
```

7.3 Setup Instructions

12. Install XAMPP and start Apache and MySQL in the XAMPP Control Panel.
13. Clone or copy the event-ticketing folder into C:\xampp\htdocs\.
14. Open http://localhost/phpmyadmin, choose Import, then import database/schema.sql followed by database/seed.sql.
15. Check api/config/config.php. The XAMPP default is user root with an empty password. Every member must use the same JWT_SECRET.
16. Open http://localhost/event-ticketing/api/ . The message "Event Ticketing & Venue Booking API is running." confirms that the backend works.
17. Point your frontend to the API base URL: http://localhost/event-ticketing/api

7.4 Git Workflow

18. The main branch must always work. After the first commit, nobody pushes directly to main.
19. Each member works on a branch, for example member2-database-users.
20. Changes are merged into main through a Pull Request reviewed by at least one other member.
21. Commit messages follow a simple style: feat(users): add DELETE /users/{id}, fix(auth): return 401 for expired token, docs: add ERD.

Commands used to create the repository (Member 2)

```
cd C:\xampp\htdocs\event-ticketing
git init
git add .
git commit -m "chore: initial project structure, database scripts and users API"
git branch -M main
git remote add origin https://github.com/<your-username>/event-ticketing.git
git push -u origin main
```

After pushing, invite the other members and the lecturer under Settings → Collaborators.

7.5 Contribution Record

Member	Main contribution	Evidence
Member 1	System overview, event browsing frontend	Commits and files in frontend/member1
Member 2	Database design, ERD, SQL scripts, Users API, JWT login, repository setup	database/*.sql, api/routes/users.php, api/routes/auth.php, api/core/*, docs/ERD.png, GitHub commits
Member 3	Events API, event management frontend, API documentation	Commits and files
Member 4	Venues API, venue frontend, Postman testing	Commits and files
Member 5	Bookings API, booking frontend, QR API, video	Commits and files

Appendix A – schema.sql (DDL)

```
-- ======================================================================
-- Event Ticketing and Venue Booking System  |  DDL (schema)
-- Database: event_ticketing_db    |  Engine: MySQL / MariaDB (XAMPP)
-- Import order: 1) schema.sql  2) seed.sql
-- ======================================================================
CREATE DATABASE IF NOT EXISTS event_ticketing_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE event_ticketing_db;


SET FOREIGN KEY CHECKS = 0;
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS events;
DROP TABLE IF EXISTS venues;
DROP TABLE IF EXISTS users;
SET FOREIGN_KEY_CHECKS = 1;


------
-- USERS  (roles: admin | organiser | customer)
------
CREATE TABLE users (
  id             INT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  name           VARCHAR(100)  NOT NULL,
  email          VARCHAR(150)  NOT NULL,
  password hash VARCHAR(255)  NOT NULL,
  role           ENUM('admin','organiser','customer') NOT NULL DEFAULT 'customer',
  phone          VARCHAR(20)    NULL,
  created_at     TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated at     TIMESTAMP      NOT NULL DEFAULT CURRENT TIMESTAMP ON UPDATE CURRENT TIMESTAMP,
  CONSTRAINT uq users email UNIQUE (email)
) ENGINE=InnoDB;


------
-- VENUES
------
CREATE TABLE venues (
  id             INT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  name        VARCHAR(150)   NOT   NULL,
  address      VARCHAR(255)   NOT  NULL,
  city        VARCHAR(100)   NOT   NULL,
  capacity       INT UNSIGNED  NOT NULL,
  price per day DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  status         ENUM('available','unavailable','maintenance') NOT NULL DEFAULT 'available',
  created_at     TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT chk venues capacity CHECK (capacity > 0)
) ENGINE=InnoDB;


------
-- EVENTS  (each event is held at one venue and owned by one organiser)
------
CREATE TABLE events (
  id                 INT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  title              VARCHAR(150)  NOT NULL,
  description        TEXT           NULL,
  category           VARCHAR(50)    NOT NULL DEFAULT 'Community',
  venue_id           INT UNSIGNED  NOT NULL,
  organiser_id       INT UNSIGNED  NOT NULL,
  event_date         DATE           NOT NULL,
  start time         TIME           NOT NULL,
  end time           TIME           NOT NULL,
  ticket price       DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  total_tickets      INT UNSIGNED  NOT NULL,
  available_tickets INT UNSIGNED  NOT NULL,
  status            ENUM('draft','published','cancelled','completed') NOT NULL DEFAULT 'draft',
  created_at         TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_events_venue       FOREIGN KEY (venue_id)       REFERENCES venues(id) ON DELETE
RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_events_organiser FOREIGN KEY (organiser_id) REFERENCES users(id)     ON DELETE
RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk events tickets  CHECK (available tickets <= total tickets),
  INDEX idx_events_date    (event_date),
```

```
  INDEX idx_events_status (status)
) ENGINE=InnoDB;


------ ---------------------------------------------------------------------------------------------
-- BOOKINGS  (a customer books tickets for one event)
------
CREATE TABLE bookings (
  id           INT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  booking_ref  VARCHAR(30)   NOT  NULL,
  user_id   INT  UNSIGNED  NOT  NULL,
  event_id   INT  UNSIGNED  NOT  NULL,
  quantity     INT UNSIGNED  NOT NULL,
  total price  DECIMAL(10,2) NOT NULL,
  status       ENUM('pending','confirmed','cancelled') NOT NULL DEFAULT 'pending',
  qr_code_url  VARCHAR(500)  NULL,           -- filled from the external QR Code API
  created_at   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated at   TIMESTAMP     NOT NULL DEFAULT CURRENT TIMESTAMP ON UPDATE CURRENT TIMESTAMP,
  CONSTRAINT uq bookings ref UNIQUE (booking ref),
  CONSTRAINT fk_bookings_user  FOREIGN KEY (user_id)  REFERENCES users(id)  ON DELETE RESTRICT
ON UPDATE CASCADE,
  CONSTRAINT fk_bookings_event FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE RESTRICT
ON UPDATE CASCADE,
  CONSTRAINT chk_bookings_qty  CHECK (quantity > 0),
  INDEX idx_bookings_user  (user_id),
  INDEX idx bookings event (event id)
) ENGINE=InnoDB;
```

Appendix B – seed.sql (DML)

```
-- ========================================================================
-- Sample data (DML)  |  Run AFTER schema.sql
-- Login accounts (password in brackets):
--   admin@event.com      (Admin@123)       -> admin
--   hafiz@event.com      (Organiser@123)  -> organiser
--   meiling@event.com    (Organiser@123)  -> organiser
--   daniel@mail.com / nurul@mail.com / arjun@mail.com (Customer@123) -> customer
-- Passwords are stored as bcrypt hashes (never plain text).
-- ========================================================================
USE event_ticketing_db;


INSERT INTO users (id, name, email, password hash, role, phone) VALUES
(1,                 'Aisyah                      Rahman',                  'admin@event.com',
'$2b$10$5bSdp6a9JEKmgz9tbPLSaub5Y58PV8mnmYK.rn6Pi/.gOHH04nu7y', 'admin',     '012-3456789'),
(2,              'Hafiz              Ismail',                             'hafiz@event.com',
'$2b$10$ZztYr1.jw1FsgQCHpnA1eOWnuGU654IFoHnt.rWOG/IXiP33N33pe', 'organiser', '013-1112233'),
(3,           'Tan            Mei           Ling',                       'meiling@event.com',
'$2b$10$ZztYr1.jw1FsgQCHpnA1eOWnuGU654IFoHnt.rWOG/IXiP33N33pe', 'organiser', '014-5556677'),
(4,          'Daniel          Lim',                                       'daniel@mail.com',
'$2b$10$y33K3c8N1yN7j0uX6hQ1Ae0NV/vNdg6RiJdMzZHwm7gtzIGDkV17i', 'customer',  '016-7778899'),
(5,          'Nurul          Huda',                                       'nurul@mail.com',
'$2b$10$y33K3c8N1yN7j0uX6hQ1Ae0NV/vNdg6RiJdMzZHwm7gtzIGDkV17i', 'customer',  '017-2223344'),
(6,            'Arjun            Kumar',                                  'arjun@mail.com',
'$2b$10$y33K3c8N1yN7j0uX6hQ1Ae0NV/vNdg6RiJdMzZHwm7gtzIGDkV17i', 'customer',  '019-8889900');

INSERT INTO venues (id, name, address, city, capacity, price_per_day, status) VALUES
(1, 'Dewan Jubli Intan',              'Jalan Petri',          'Muar',          800,  3500.00,
'available'),
(2, 'Muar Trade Centre Hall A',      'Jalan Junid',          'Muar',          1500, 6000.00,
'available'),
(3, 'Danga City Convention Centre',  'Jalan Skudai',         'Johor Bahru',  3000, 12000.00,
'available'),
(4, 'Stadium Tertutup Larkin',         'Jalan Mutiara Emas', 'Johor Bahru',  5000, 20000.00,
'available'),
(5, 'Tanjung Emas Open Arena',         'Jalan Tanjung Emas', 'Muar',          2000, 4500.00,
'maintenance');

INSERT INTO events (id, title, description, category, venue_id, organiser_id, event_date,
start_time, end_time, ticket_price, total_tickets, available_tickets, status) VALUES
(1, 'Muar Music Fest 2026',  'A night of live local bands and food stalls.', 'Music',       2,
2, '2026-11-14', '18:00:00', '23:00:00', 60.00,  1000, 994, 'published'),
(2, 'Johor Tech Summit 2026', 'Talks and workshops on AI, cloud and startups.', 'Technology',
3, 3, '2026-12-05', '09:00:00', '17:00:00', 120.00, 300,  297, 'published'),
(3, 'Charity Gala Dinner',   'Fundraising dinner with live auction.', 'Charity',           1,
2, '2026-11-28', '19:00:00', '22:30:00', 45.00,  500,  500, 'published'),
(4,   'Stand-Up   Comedy   Night',      'Top   comedians   perform   live.',   'Entertainment',
1, 3, '2026-12-12', '20:00:00', '22:00:00', 80.00,  200,  200, 'cancelled'),
(5, 'Startup Pitch Day',     'Student and early-stage startups pitch to investors.', 'Business',
2, 3, '2027-01-16', '10:00:00', '16:00:00', 35.00,  150,  148, 'published'),
(6, 'Futsal Charity Cup',    'Inter-school futsal tournament.', 'Sports',                  4,
2, '2027-02-07', '08:00:00', '18:00:00', 25.00,  400,  395, 'published');

INSERT INTO bookings (id, booking_ref, user_id, event_id, quantity, total_price, status,
qr_code_url) VALUES
(1, 'BK-20261001-0001', 4, 1, 2, 120.00, 'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261001-0001'),
(2, 'BK-20261001-0002', 5, 1, 4, 240.00, 'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261001-0002'),
(3, 'BK-20261002-0003', 6, 2, 1, 120.00, 'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261002-0003'),
(4, 'BK-20261003-0004', 4, 3, 3, 135.00, 'pending',    NULL),
(5, 'BK-20261003-0005', 5, 4, 2, 160.00, 'cancelled', NULL),
(6, 'BK-20261004-0006', 6, 5, 2, 70.00,  'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261004-0006'),
(7, 'BK-20261005-0007', 4, 2, 2, 240.00, 'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261005-0007'),
(8, 'BK-20261006-0008', 5, 6, 5, 125.00, 'confirmed', 'https://api.qrserver.com/v1/create-qr-
code/?size=200x200&data=BK-20261006-0008');
```

8 Additional and Advanced Features

8.1 Event Search, Filtering, and Pagination

The list endpoint can search titles and descriptions and filter by category, venue, status, and organiser. It returns the current page, page size, total result count, and number of pages. The page size is capped at 100 to keep responses bounded.

8.2 Role-Based Access and Ownership

Guests and customers can browse published events. Organisers and admins can also view drafts, cancelled events, and completed events. All write operations require a JWT. An organiser can update or delete only events assigned to their own account, while an admin can manage events across organisers. Organiser reassignment is restricted to admins and validated against the target user's role.

8.3 Venue Availability and Schedule Conflict Checks

Before an event is created, the API checks that its venue exists and has available status. It then checks for a draft or published event at the same venue, on the same date, with overlapping times. A conflict returns HTTP 409 and includes the conflicting event details. The same checks run when an update changes an event's venue, date, time, or status into a schedulable state.

8.4 Input Validation and Booking-Safe Deletion

The API validates date and time formats, event status, text lengths, positive ticket totals, non- negative available tickets, and ticket price precision. It also ensures available tickets do not exceed total tickets and the end time follows the start time. Deletion is blocked if confirmed bookings exist; the database foreign key protects any event with remaining linked bookings. These checks return consistent JSON errors rather than raw database messages.

8.5 Event Management Frontend

The Member 3 frontend in frontend/member3-event-management/ provides public event browsing and role-aware event management. Visitors can search, filter, paginate, and open event details. Admins and organisers can use forms to create and edit events, and can delete eligible events. Venue choices are loaded from the shared Venues API. The page stores the selected API URL and login session in browser storage and can display API validation and conflict messages.

8.6 API Verification Evidence

The PHP route passed php -l, and the frontend JavaScript passed node --check. These are syntax checks only. Runtime requests against the shared XAMPP/MySQL database have not yet been completed in this workspace. Capture Postman or browser evidence for the following before submitting the final combined report:

Scenario	Expected result	Evidence status
GET published event list as guest	200 with published events and pagination	Capture during local API run
POST valid event as organiser	201 with created event	Capture during local API run
POST invalid event data	422 with field errors	Capture during local API run
POST event into unavailable or overlapping venue schedule	409 conflict	Capture during local API run
Update another organiser's event	403 for an organiser	Capture during local API run
Delete event with linked booking	409 conflict	Capture during local API run

9.0 Debugging and Performance Optimisation

Debugging was carried out during API development and frontend integration. Postman responses were used to identify authentication, validation and database connection issues. The following table summarises the issues encountered and the solutions applied.

Issue	Cause	Solution
Database connection failed	MySQL used port 3307, but the PDO connection did not specify this port.	Added DB_PORT to the PDO connection string and matched the configuration to XAMPP.
API returned 401 when a token had expired	The JWT was no longer valid.	Logged in again to obtain a fresh token. Login scripts were added to Postman to populate token variables automatically.
Postman displayed “Secrets Detected” when saving requests	A JWT value was entered directly into the request.	Replaced literal tokens with {{admin_token}} and {{customer_token}}. Token values were cleared before exporting.
Update and delete requests depended on a fixed test ID	Newly created venues received different auto-increment IDs.	Added a post-response script to store the created ID in venue_id, then used {{venue_id}} in subsequent requests.

Responses such as 400, 403, 404, 409 and 422 were expected outcomes for invalid or restricted requests. These responses demonstrated correct error handling rather than unresolved system faults.

9.2 Performace Testing

The response time of GET /venues was measured using Postman on the local XAMPP server. After a warm-up request, five requests were sent sequentially, and the response time of each request was recorded.

Request	Response Time
1	44 ms
2	54 ms
3	38 ms
4	40 ms
5	49 ms
Average	45 ms
Minimum	38 ms
Maximum	54 ms

The average response time was 45 ms, with readings ranging from 38 to 54 ms. These results describe a small local test using the sample database. They do not measure performance under concurrent users or demonstrate an improvement from a before-and-after optimisation comparison.

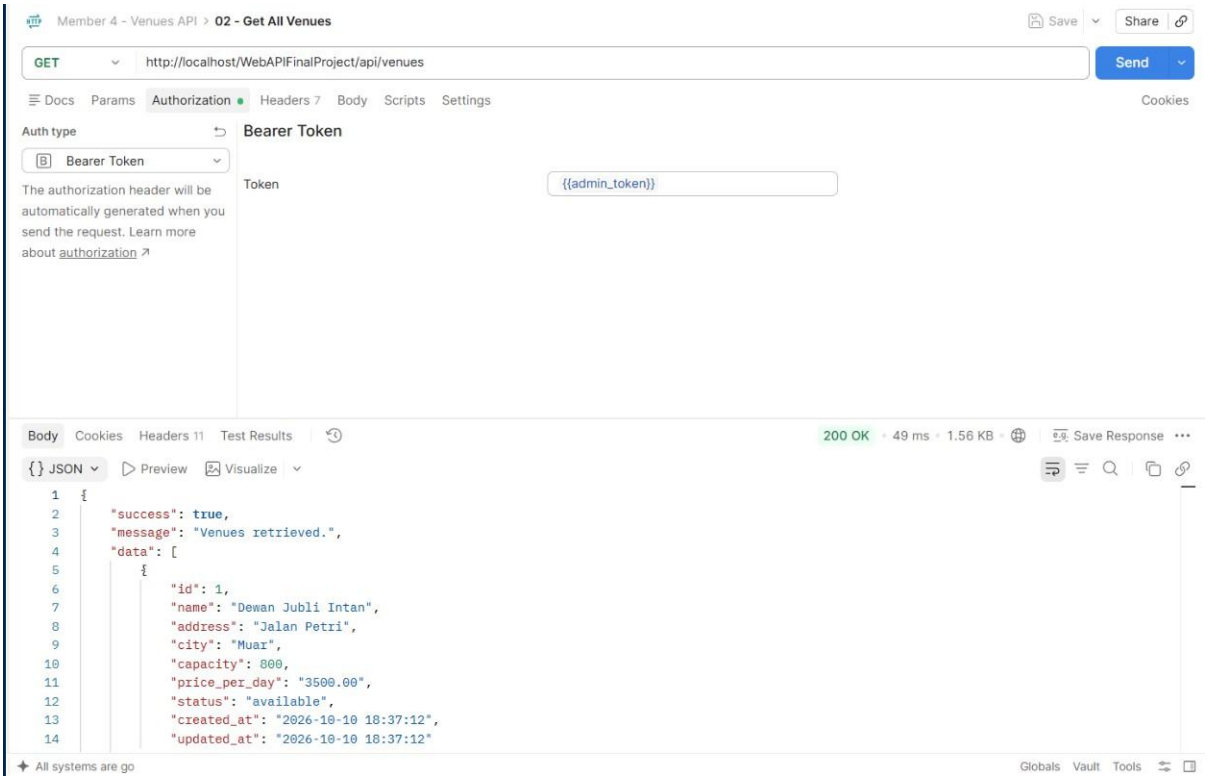


Figure 9.1: Postman response status and response time for GET /venues.

9.3 Implemented Improvements and Future Optimisation

The following improvements were implemented to support reliable API and frontend operation:

Improvement	Purpose
Server-side input validation	Rejects invalid venue details before database operations.
Prepared SQL statements	Keeps user-supplied values separate from SQL commands.
JWT authentication and role checks	Restricts venue management operations to authorised users.
Foreign-key error handling	Returns a clear 409 response when a venue is linked to events.
Availability conflict checking	Identifies overlapping draft or published events and checks venue status.
Disabled buttons during requests	Reduces repeated submissions while a request is processing.
Local frontend search	Filters the loaded venue list without sending an API request for each search input.
Postman token and venue ID variables	Makes the test sequence easier to repeat.

These changes improve validation, error handling and testing reliability. Their effect on response time was not measured through a before-and-after comparison.

For larger datasets, future optimisation could include server-side pagination and filtering. A composite index on event availability query fields could also be evaluated using EXPLAIN and repeatable performance tests. Concurrent-user testing would be required to assess performance under greater load.

Venue availability checking does not reserve a time slot. The Events API should enforce availability rules during event creation and updates to prevent conflicting schedules.

10.0 Individual Reflection and Contribution Record

10.1 Nurbasyierah Reflective Report

Throughout the development of the Event Ticketing and Venue Booking System, I was responsible for developing the Event Browsing frontend and preparing the System Overview section of the group report. My main contribution was creating an interactive interface that allows users to browse available events, search for specific events, filter event categories, and view detailed event information.

For the frontend development, I used PHP, HTML, CSS, and JavaScript to create a responsive and user-friendly website. I also developed a GET API endpoint using PHP and MySQL to retrieve event information from a database. This allowed the frontend to display actual database records instead of relying on hardcoded event data.

Additionally, I created a local development database with sample event and venue records to test the connection between the frontend and backend. I used Postman to test five API requests, including successful data retrieval, invalid event IDs, unavailable event records, and unsupported HTTP methods. These tests helped me understand how HTTP status codes and JSON responses are used in RESTful API development.

One challenge I faced was connecting the frontend to the PHP API and ensuring the information displayed correctly after database changes. I addressed this by checking the API responses and testing the website after updating the database.

Overall, this project improved my understanding of RESTful APIs, database integration, API testing, and frontend development. It also helped me develop better debugging and problem-solving skills.

10.2 Nur Fadilah Reflective Report

In this project I was responsible for the database design, the Users API, login with JWT authentication, and the shared GitHub repository. I designed a MySQL database with four related tables (Users, Venues, Events and Bookings), wrote the SQL scripts with sample data, and drew the ERD. I then built the Users API with full CRUD, pagination, search and role-based access control for administrator, organiser and customer. I also set up my own frontend with role-based menus, and tested my endpoints in Postman with both success and error cases.

I faced several problems. First, I had to run the project on Laragon instead of XAMPP, so I learned to place the folder in the www directory and use the correct API URL. Second, my Git push failed because I used a placeholder repository address, and later I found that my group used a different repository from the one I created. I solved this by cloning the group repository and copying my frontend into it. Third, my Postman requests returned 401 errors because I pasted an incomplete token, which taught me that JWT tokens must be copied fully and expire after two hours. Finally, my frontend showed "Resource not found" for events, which showed me that my pages depend on my teammates' APIs.

From this project I learned how REST APIs, JWT authentication, foreign keys and Git collaboration work together. I also learned how important it is to agree on the repository and response format early. If I did it again, I would agree on the repository and API format at the start, add automated tests, and add features such as rate limiting and logging.

10.3 NurSyakirin Reflective Report

As Member 4 of the Event Ticketing and Venue Booking System project, I was responsible for developing the Venues API and Venue Management frontend. I implemented venue creation, retrieval, updating and deletion, together with availability checking based on venue status and overlapping event schedules. I also prepared API testing evidence, exported the Postman collection, and documented debugging and performance results.

One challenge was connecting the backend to my local database because MySQL used port 3307. I resolved this by adding the correct port to the configuration and PDO connection string. Expired JWT tokens also interrupted testing, so I obtained fresh tokens and configured Postman scripts to store them automatically. When GitHub rejected my pushes because the remote repository contained newer changes, I integrated those changes before pushing again.

Through this project, I learned how a JavaScript frontend communicates with a PHP REST API through HTTP requests and JSON responses. I also improved my understanding of authentication, role-based permissions, input validation and appropriate HTTP status codes. Testing both successful and unsuccessful requests helped me verify that the API handled different situations correctly.

Evidence of my contribution includes Git commits, api/routes/venues.php, the Venue Management frontend files, the exported Postman collection, testing screenshots, API documentation and my report. Five local requests to retrieve venues recorded an average response time of 45 milliseconds.

For future improvements, I would introduce automated API assertions, server-side pagination and more integration testing. I would also test concurrent requests and coordinate earlier with teammates to ensure availability rules are consistently enforced during event creation and updates.

10.4 Muhammad Syamir Reflective Report

As Member 3 for the Event Ticketing and Venue Booking System project, my primary responsibilities encompassed the development of the Events API, the Event Management Frontend, and the compilation of comprehensive technical documentation. I authored Section 3.0 of our report, which established the RESTful API documentation, detailing the API URLs, HTTP methods, JSON request and response structures, HTTP status codes, and JWT authentication requirements. Additionally, I documented our system's advanced features in Section 8.0. To ensure my API design aligned seamlessly with our initial system objectives and database models, I regularly cross-referenced the foundational project requirements outlined in the part 1.docx file.

My core coding contribution involved building the Events API, which handles full CRUD (Create, Read, Update, Delete) operations for the system. I integrated this backend with the Event Management Frontend I developed, allowing authorised users to seamlessly add, view, update, and delete events through a responsive web interface. A major focus of my work was implementing middleware and advanced API features. I successfully integrated role-based access control into the Events API, ensuring that administrators can manage all events, organisers can only modify events they own, and customers or guests are restricted strictly to browsing published events. I also built out advanced search, filtering, and pagination logic to handle large volumes of event data efficiently.

One of the primary challenges I faced during development was ensuring seamless communication between the frontend and the secured API, particularly when handling complex validation states. Implementing booking-safe deletion and venue availability checks required careful database querying to prevent data integrity issues, such as accidentally deleting an event that already had confirmed bookings. When these edge cases occurred, I had to ensure the API returned appropriate HTTP status codes, such as 409 Conflict or 422 Unprocessable Entity, and that the frontend gracefully parsed these JSON responses to display user-friendly error messages rather than exposing raw database faults. Ultimately, this project significantly strengthened my understanding of RESTful architecture, secure API endpoint design, and front-to-back system integration, while building out the middleware taught me how to effectively modularise security checks rather than duplicating code across multiple routes.

10.5 Muhammad Arief Azim Reflective Report

As Member 5 of the development team for the EventHub ticketing and management system, my primary technical responsibility centered on designing and implementing the Booking Management and Digital Ticket Verification subsystem. Beyond individual module development, my role extended across the wider project scope to include documenting the overall system implementation, writing the combined system demonstration narratives, and compiling the final group demonstration video. This report critically reflects on my technical execution, the architectural challenges faced during backend and frontend integration, team collaboration dynamics, and personal learnings gained throughout the engineering process.

On the backend, my main focus was constructing the booking handling routes within our PHP RESTful API structure. A critical technical challenge involved ensuring data integrity during ticket reservations, where seat allocation directly impacts event inventory. In early testing, a standard lookup query followed by a separate update statement exposed the system to potential race conditions under simultaneous client requests. To guarantee atomic execution and prevent negative seat counts, I implemented explicit MySQL database transactions using PHP Data Objects. Wrapping the seat availability check, booking record insertion, and seat count deduction inside explicit transaction start and commit boundaries ensured that any partial failure automatically rolled back the entire operation, preserving database consistency.

Translating these backend capabilities to the frontend required seamless integration with the authentication subsystem built by Member 2. Every protected action, such as creating or cancelling a reservation, required extracting the JSON Web Token stored in browser storage and appending it to the authorization request headers. Managing these asynchronous operations in JavaScript required structured error handling and modern async-await syntax to gracefully handle potential authorization failures, such as expired sessions, without causing client-side errors or requiring full page reloads. This approach maintained a responsive user experience while ensuring that the booking dashboard stayed synchronized with backend database updates.

For digital ticket issuance, I integrated a third-party image generation API to render scannable entry passes. Instead of generating and storing static image files on the local web server—which would accumulate unnecessary storage overhead over time—the client-side script serializes key booking parameters, including the reference code, attendee identifier, and seat count, into a URL-encoded JSON payload. This payload is passed dynamically to the external API, rendering the scannable code within a modal overlay on demand. While this approach minimized server storage requirements and simplified frontend rendering, relying on an external service introduced a third-party dependency where service downtime would prevent ticket display, highlighting an important trade-off between architectural simplicity and system autonomy.

Beyond core coding, coordinating the shared implementation and demonstration sections required a comprehensive understanding of the entire system stack. I reviewed the codebases of all four team members to accurately articulate how our shared database handlers, authorization middleware, and standardized response utilities interacted with each module. Compiling the final group demonstration video added another layer of technical coordination, as I had to standardize disparate screen recording resolutions into a uniform layout, balance varying microphone audio levels for consistency, and edit the raw footage into a cohesive chronological narrative tracing the user journey from public event discovery through to ticket verification.

Reflecting on the development lifecycle, the implementation succeeded in delivering a functional, secure reservation pipeline with clear API boundaries and proper transaction handling. However, under heavy production loads, the concurrency management could be

further strengthened by implementing explicit row-level locking during availability checks, and replacing the external QR API with a native server-side library would eliminate external dependency risks. Overall, this project deepened my practical engineering skills in transaction processing, API middleware design, client-side state management, and technical project management, providing valuable insights that will directly inform my approach to future full-stack software development.

Github Repository

https://github.com/syakirin1908/WebAPIFinalProject

Video Demonstration Link

https://drive.google.com/drive/folders/18Vq9-WTMaq30gqKecCtIrVPM8BZPXry5?usp=sharing